

Bachelor Thesis

im Studiengang Geoinformatik

zur Erlangung des Grades eines
Bachelor of Science (B.Sc.)

über das Thema

LLM-gestützte Generierung von Playwright-E2E-Tests aus Use Cases

Einfluss von UI-Kontext am Beispiel von WebGIS-Anwendungen

von

Nils Große Beikel

Betreuer (Universität):	Dr. Christian Knoth
Betreuer (Unternehmen):	Dr. Thore Fechner
Matrikelnummer :	546548
Abgabedatum :	31. August 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung und Erkenntnisinteresse	1
1.3 Aufbau der Arbeit	1
2 Grundlagen	2
2.1 Large Language Models	2
2.1.1 Funktionsprinzip und Kontext als Qualitätsfaktor	2
2.1.2 Einsatz im Software Engineering	3
2.2 E2E-Testautomatisierung mit Playwright	4
2.2.1 Teststufen und Einordnung von E2E-Tests	4
2.2.2 Playwright: Selektoren, Assertions, asynchrone UIs	5
2.3 Open Pioneer Trails	6
2.3.1 Framework-Überblick und Komponentenmodell	6
2.3.2 data-testid-Konventionen und Testbarkeit	7
2.4 WebGIS-Anwendungen und Testkomplexität	8
3 Stand der Forschung	10
3.1 LLM-gestützte Testgenerierung	10
3.2 Forschungslücke und Einordnung dieser Arbeit	11
4 Methodik und Forschungskonzept	13
4.1 Überblick und Forschungsdesign	13
4.2 Use Cases: Definition und Format	15
4.3 Versuchsaufbau: Die vier Kontextstufen	16
4.3.1 Stufe 1: Baseline (kein UI-Kontext)	16
4.3.2 Stufe 2: Automatischer Accessibility-Snapshot	16
4.3.3 Stufe 3: Automatisch generierte UI-Map	17
4.3.4 Stufe 4: Manuell erstellte UI-Map	17
4.3.5 Bonus-Stufe 5: Self-Improvement-Loop	18
4.3.6 Erwartungen	19

4.4	LLM-Konfiguration (Modell, Temperatur, Prompt-Struktur)	19
4.5	Evaluationsverfahren	20
4.5.1	Phase 1: Deterministische Ausführung	20
4.5.2	Phase 2: LLM-as-a-Judge	21
5	Implementierung der Evaluationsumgebung	24
5.1	Demo-WebGIS-Anwendung	24
5.2	Use-Case-Sammlung	25
5.3	Manuell erstellte Referenztests	27
6	Durchführung und Evaluation	30
6.1	Durchführung des Experiments	30
6.2	Ergebnisse der Ausführung (Phase 1)	30
6.3	Ergebnisse der Qualitätsbewertung (Phase 2)	33
6.3.1	Kontrolle der maschinellen Bewertung	33
6.4	Ergebnisse der Bonus-Stufe 5	33
6.5	GIS-spezifische Besonderheiten	33
7	Diskussion	34
7.1	Interpretation der Ergebnisse	34
7.2	Limitationen	34
7.3	Handlungsempfehlungen	34
8	Fazit und Ausblick	34
	Anhang	35
	Literaturverzeichnis	39

Abbildungsverzeichnis

Abbildung 1: Testpyramide nach Mike Cohn	4
Abbildung 2: Open Pioneer Trails Paketökosystem	7
Abbildung 3: Ausprägungen der Variable UI-Kontext	14
Abbildung 4: Generierungs- und Evaluationspipeline	15
Abbildung 5: Self-Improvement-Loop (Stufe 5)	18
Abbildung 6: Demo-WebGIS-Anwendung	24
Abbildung 7: Ausführungskategorien je Use Case und Stufe	32

Tabellenverzeichnis

Tabelle 1: Übersicht der Use Cases mit Komplexität und getesteter Kernfunktion . .	26
Tabelle 2: Ausführungsergebnisse je Stufe	31
Tabelle 3: Skalenstufen der vier Bewertungsdimensionen	35

Abkürzungsverzeichnis

API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
CSS	Cascading Style Sheets
DOM	Document Object Model
DWD	Deutscher Wetterdienst
E2E	End-to-End
EPSG	European Petroleum Survey Group
GIS	Geoinformationssystem
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
OPT	Open Pioneer Trails
PDF	Portable Document Format
PNG	Portable Network Graphics
PNPM	Performant Node Package Manager
REST	Representational State Transfer
UI	User Interface
URL	Uniform Resource Locator
WGS84	World Geodetic System 1984
WMS	Web Map Service

1 Einleitung

1.1 Motivation und Problemstellung

1.2 Zielsetzung und Erkenntnisinteresse

1.3 Aufbau der Arbeit

2 Grundlagen

2.1 Large Language Models

Large Language Models (LLMs) sind maschinelle Lernmodelle, die auf sehr großen Textmengen trainiert werden, um natürliche Sprache zu verarbeiten und zu erzeugen. Das Attribut „large“ bezieht sich dabei sowohl auf den Umfang der Trainingsdaten als auch auf die Anzahl der Modellparameter, die häufig im Bereich mehrerer Milliarden liegt. Technisch beruhen sie auf künstlichen neuronalen Netzen, genauer gesagt auf der Transformer-Architektur (*Hou et al., 2024*).

2.1.1 Funktionsprinzip und Kontext als Qualitätsfaktor

Moderne LLMs basieren auf der Transformer-Architektur, die von *Vaswani et al. (2017)* eingeführt wurde. Ihr zentraler Bestandteil ist der Self-Attention-Mechanismus. Er erlaubt es dem Modell, die Beziehungen zwischen allen Elementen einer Eingabe unabhängig von ihrer Position zu gewichten. Anders als bei früheren Ansätzen wird die Eingabe dadurch nicht sequentiell, sondern weitgehend parallel verarbeitet, was das Training auf sehr großen Textmengen erst praktikabel gemacht hat (*ebd.*). Vor der Verarbeitung wird der Text in sogenannte Tokens zerlegt. Tokens sind in der Regel keine ganzen Wörter, sondern Teilwörter oder einzelne Zeichen, auf denen das Modell dann ausschließlich arbeitet.

Die Texterzeugung erfolgt autoregressiv. Auf Basis der bereits vorliegenden Tokens berechnet das Modell für das nächste Token eine Wahrscheinlichkeitsverteilung über das gesamte Vokabular. Aus dieser Verteilung wird ein Token ausgewählt und an die Eingabe angehängt. Der Vorgang wiederholt sich, bis die Ausgabe abgeschlossen ist. Wie stark die Auswahl vom jeweils wahrscheinlichsten Token abweichen darf, steuert unter anderem der Temperatur-Parameter. Ein niedriger Wert führt zu deterministischeren, ein hoher zu variableren Ausgaben (*ebd.*).

Alle Tokens, die das Modell für eine Generierung berücksichtigt, befinden sich im sogenannten Kontextfenster. Dieses umfasst die Eingabe (den Prompt) und die bereits erzeugte Ausgabe und ist in seiner Größe begrenzt. Wissen, das nicht in den Modellgewichten verankert ist, also nicht während des Trainings gelernt wurde, muss dem Modell zur Laufzeit über dieses Fenster bereitgestellt werden (*Brown et al., 2020*).

Eng damit verbunden ist die Fähigkeit zum In-Context Learning. *Brown et al. (2020)* zeigten, dass große Sprachmodelle Aufgaben allein anhand von Anweisungen oder Beispielen

im Prompt lösen können, ohne dass die Modellgewichte angepasst werden. Das Modell „lernt“ hierbei nicht im klassischen Sinne, sondern nutzt die im Kontext bereitgestellten Informationen direkt als Grundlage für seine Vorhersage. Welche Inhalte im Prompt stehen, bestimmt damit maßgeblich die Ausgabe.

Daraus folgt ein zentraler Punkt für diese Arbeit. Ein LLM hat kein konkretes Wissen über eine individuell entwickelte Benutzeroberfläche. Informationen über die tatsächlich vorhandenen UI-Elemente, deren Selektoren und Zustände stehen weder in den Trainingsdaten noch sind sie zur Laufzeit zugänglich, solange sie nicht explizit im Prompt bereitgestellt werden. Die Qualität des generierten Testcodes hängt damit unmittelbar von Qualität und Umfang des bereitgestellten Kontexts ab.

2.1.2 Einsatz im Software Engineering

LLMs werden zunehmend im Software Engineering eingesetzt, insbesondere zur Generierung von Quellcode. Sichtbar wird das an Werkzeugen wie GitHub Copilot, das auf Grundlage des umgebenden Quellcode-Kontexts Vorschläge zur Codevervollständigung erzeugt und direkt in die Entwicklungsumgebung integriert ist. Der systematische Literaturüberblick von *Hou et al. (2024)* zeigt, dass der Einsatz von LLMs im Software Engineering inzwischen breit erforscht wird und deutlich über die reine Codevervollständigung hinausreicht.

Ein Anwendungsfeld ist die automatisierte Testgenerierung. LLMs werden hier eingesetzt, um aus natürlichsprachlichen oder strukturierten Beschreibungen Unit-Tests, Akzeptanztests und End-to-End (E2E)-Tests zu erzeugen. Bestehende Ansätze beziehen sich überwiegend auf generische Webanwendungen (*Celik, Mahmoud, 2025; Ferreira et al., 2025*). GIS-spezifische Anwendungen werden aktuell kaum berücksichtigt. Eine ausführliche Betrachtung des Forschungsstands folgt in Kapitel 3.

Dieser Möglichkeit stehen bekannte Grenzen gegenüber. Da LLMs ihre Ausgaben auf Basis von Wahrscheinlichkeiten erzeugen, können sie plausibel wirkende, faktisch aber nicht existierende Inhalte produzieren, sodass bei der Testgenerierung etwa erfundene Selektoren oder Aufrufe nicht vorhandener Funktionen genutzt werden. Dieses Phänomen wird als Halluzination bezeichnet (*Hou et al., 2024*). Hinzu kommt das fehlende Wissen über die konkrete Anwendung und der Umstand, dass das Modell keinen Zugriff auf den tatsächlichen Laufzeitzustand der Oberfläche hat. Gerade für E2E-Tests, die gezielt auf konkrete User Interface (UI)-Elemente zugreifen müssen, sind diese Einschränkungen besonders relevant.

Grundsätzlich lassen sich Sprachmodelle auf zwei Wegen an eine spezifische Aufgabe anpassen: durch Fine-Tuning, also das Nachtrainieren der Modellgewichte auf aufgabenspezifischen Daten, oder durch Prompting, bei dem das Verhalten allein über die Gestaltung der Eingabe gesteuert wird. Prompting beruht auf dem in Abschnitt 2.1.1 beschriebenen In-Context Learning von *Brown et al. (2020)*. Diese Arbeit verfolgt durchgängig den Prompting-Ansatz, da er gegenüber Fine-Tuning sowohl einen geringeren Ressourcenaufwand erfordert als auch eine bessere Generalisierbarkeit bietet. Ein prompt-basierter Workflow lässt sich ohne erneutes Training auf andere Anwendungen und Use Cases übertragen, während ein feinjustiertes Modell weitgehend auf seinen Trainingskontext beschränkt bliebe.

2.2 E2E-Testautomatisierung mit Playwright

2.2.1 Teststufen und Einordnung von E2E-Tests

Softwaretests lassen sich nach ihrem Umfang und ihrer Nähe zum Nutzerverhalten in verschiedene Stufen einteilen. Eine verbreitete Einteilung von Softwaretests ist die Testpyramide, die von *Poenisch, 2022* beschrieben wird. Sie unterscheidet drei Ebenen, wie in Abbildung 1 dargestellt. Ganz unten stehen Unit-Tests, die einzelne Funktionen oder Komponenten prüfen. Sie laufen schnell und werden in großer Anzahl geschrieben. Darüber liegen die Integrationstests, die das Zusammenspiel mehrerer Komponenten betrachten. An der Spitze stehen E2E-Tests, die die Anwendung als Ganzes aus Sicht der Nutzer prüfen.

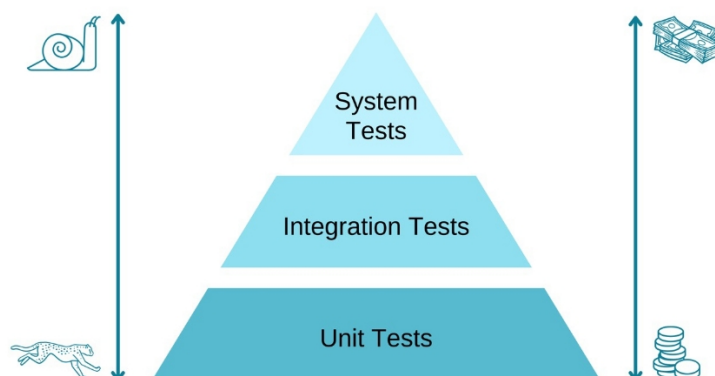


Abbildung 1: Die Testpyramide nach Mike Cohn (Quelle: ebd.)

Ein E2E-Test steuert dazu einen Browser und führt dieselben Schritte aus wie ein Mensch. Dazu gehören auf Schaltflächen klicken, Felder ausfüllen und durch die Oberfläche

navigieren. Geprüft wird, ob am Ende das erwartete Ergebnis eintritt. Dadurch geben E2E-Tests eine gute Auskunft darüber, ob eine fachliche Anforderung tatsächlich umgesetzt ist. Allerdings sind sie langsam, aufwendig zu erstellen und vergleichsweise anfällig und können schon bei kleinen Änderungen an der UI fehlschlagen. Deshalb wird empfohlen, ihre Anzahl, entsprechend der Form der Pyramide, gering zu halten.

Für WebGIS-Anwendungen sind E2E-Tests besonders relevant. Viele Probleme zeigen sich erst im Zusammenspiel der Komponenten und Interaktionen. Dazu gehört das asynchrone Laden von Geodaten, das Rendern der Karte oder die Auswirkungen der Layersteuerung auf die Kartenanzeige. Solche Abläufe lassen sich mit Unit-Tests kaum abbilden, weil sie den vollständigen, im Browser laufenden Zustand voraussetzen.

2.2.2 Playwright: Selektoren, Assertions, asynchrone UIs

Playwright ist ein Open Source E2E-Test-Framework von Microsoft zur Automatisierung von Browsern. Es steuert die Browser-Engines Chromium, Firefox und WebKit über eine einheitliche Programmierschnittstelle und führt Tests in einem echten Browser aus. Die Tests können JavaScript, TypeScript, Python, Java und -NET geschrieben, das Playwright unterstützt *Microsoft, 2026*.

Um mit der Benutzeroberfläche zu interagieren, muss ein Test die jeweiligen Elemente finden. Playwright nutzt dafür sogenannte Lokatoren. Empfohlen wird, Elemente über nutzerseitige Eigenschaften anzusprechen, etwa über ihre ARIA-Rolle oder ihren sichtbaren Text (`getByText`, `getByRole`). Solche Selektoren sind robuster gegenüber Änderungen am Markup als Cascading Style Sheets (CSS)- oder XPath-Ausdrücke, die an die konkrete Document Object Model (DOM)-Struktur gebunden sind. Eine Alternative sind Test-IDs. Über das Attribut `data-testid` lässt sich ein Element gezielt für Tests markieren und mit `getByTestId` ansprechen, unabhängig von Darstellung und Position. Welches Attribut als Test-ID gilt, ist konfigurierbar ebd.

Ein wesentliches Merkmal von Playwright ist das automatische Warten. Vor jeder Aktion prüft das Framework, ob das Zielelement sichtbar, stabil und bedienbar ist, ansonsten wird bis zu einem Timeout gewartet. Dadurch müssen nicht für jede Interaktion manuelle Wartezeiten eingebaut werden, was eine häufige Ursache für instabile Tests beseitigt. Dasselbe Prinzip gilt für Assertions. Mit `expect` formulierte Zusicherungen wie `expect(locator).toBeVisible()` oder `toHaveText()` werden so lange wiederholt, bis die Bedingung erfüllt ist oder die Zeit abläuft. Eine Assertion prüft damit nicht einen Momentzustand, sondern einen Zustand, der sich auch verzögert einstellen darf.

Für Fälle, die das automatische Warten nicht abdeckt, stellt Playwright extra Wartemethoden bereit, etwa auf eine bestimmte Netzwerkantwort (`waitForResponse`) oder einen Ladezustand der Seite (`waitForLoadState`). Das ist gerade bei WebGIS-Anwendungen wichtig, in denen Kartenkacheln und Geodaten asynchron über das Netz geladen werden und erst danach sichtbar auftauchen. Ein Test, der zu früh prüft, würde fehlschlagen, obwohl die Anwendung korrekt arbeitet. Gerade beim asynchronen Laden von Kartenkacheln und Geodaten greift damit beides ineinander: das automatische Warten für DOM-Elemente und gezielte Wartepunkte auf Netzwerkebene.

2.3 Open Pioneer Trails

2.3.1 Framework-Überblick und Komponentenmodell

Open Pioneer Trails ist ein Open Source Framework zur Entwicklung webbasierter GeoIT-Anwendungen. Es ist als gemeinsames Projekt der con terra und 52°North im Rahmen der Open-Pioneer-Initiative entstanden und steht unter der Apache 2.0 Lizenz frei auf GitHub zur Verfügung. Trails ist ein Entwicklungsframework und kein fertiges Produkt. Anwendungen werden also nicht konfiguriert, sondern programmiert, wobei das Framework wiederverwendbare Komponenten und Vorlagen bereitstellt *52°North GmbH, Matthes Rieke Managing Director, 2026*.

Technisch besteht Trails aus modernen Webtechnologien. Die Anwendungen werden in TypeScript mit React geschrieben, das Styling basiert auf Chakra UI, als Kartenkomponente wird OpenLayers verwendet, die Paketverwaltung läuft über Performant Node Package Manager (PNPM) und als Build-System wird Vite verwendet *Open Pioneer, 2026*.

Eine Open Pioneer Trails-Anwendung ist auf Paketen aufgebaut. Ein Paket bündelt React-Komponenten, Services, Übersetzungen und Styling. Die Anwendung selbst ist ebenfalls ein Paket und wird zu einer Web Component gebaut, dadurch kann sie einfach in bestehende Web-Anwendungen integriert werden. Aktuell werden zwei Gruppen wiederverwendbarer Pakete bereitgestellt. Die Core Packages enthalten kartenlose Basisfunktionen wie Authentifizierung, Notifier, Internationalisierung mit I18n und einen Hypertext Transfer Protocol (HTTP)-Service. Die OpenLayers Base Packages bauen darauf auf und liefern die Kartenkomponenten, darunter Layerübersicht, Kartennavigations-Werkzeuge, Legende, Selektion und Ergebnisliste, Suche, Messwerkzeug, Koordinatenanzeige und Kartendruck. Abbildung 2 gibt einen Überblick über das Paketökosystem.

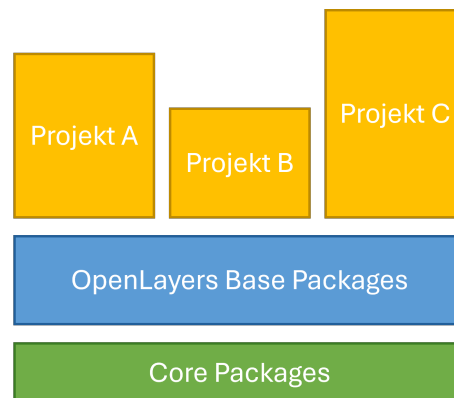


Abbildung 2: Das Open Pioneer Trails Paketökosystem

Die Anbindung der Karte erfolgt nicht direkt über OpenLayers, sondern über eine eigene Schnittstelle. Diese erweitert das Datenmodell, vereinfacht den Umgang mit dem Kartenzustand und stellt ihn als TypeScript-Schnittstelle (MapModel) bereit. Der direkte Zugriff auf die OpenLayers-Application Programming Interface (API) bleibt bei Bedarf möglich 52 North GmbH, Arne Vogt, 2025. Für die Demo-Anwendung relevant sind unter anderem die Pakete für Karte, Layerübersicht, Basemap-Auswahl, Suche, Messen und Druck.

Trails richtet sich explizit an Entwickler und erfordert Programmierkenntnisse. Dafür lässt es Layout und Funktionalität frei gestalten und bringt typische Geoinformationssystem (GIS)-Widgets als fertige Komponenten mit. Für die Erstellung einer spezifischen Demo-WebGIS-Anwendung im Rahmen dieser Arbeit ist es damit eine passende Grundlage.

2.3.2 data-testid-Konventionen und Testbarkeit

Ein verbreitetes Mittel, um Elemente in Tests gezielt anzusprechen, ist das Hypertext Markup Language (HTML)-Attribut `data-testid`. Es wird ausschließlich für Tests vergeben und ist von Styling und DOM-Struktur unabhängig. Ein Test, der ein Element über seine Test-ID anspricht, bleibt auch dann stabil, wenn sich CSS-Klassen oder Strukturen ändern. Playwright kann dann wie in Abschnitt 2.2.2 beschrieben mit `getByTestId` darauf zugreifen.

Trails unterstützt diese Konvention standardmäßig. Alle öffentlichen Komponenten teilen sich die gemeinsame Schnittstelle `CommonComponentProps`, die optional eine `data-testid` entgegennimmt und an das DOM weiterreicht. Bei allen anderen Elementen von Chakra UI werden nur Chakra-bezogene Style-Props herausgefiltert und alles andere ans DOM übergeben, darunter auch die Test-ID Chakra Systems, 2026. Den Wert gibt dabei die

Entwicklung vor, einen automatischen Standardwert vergibt das Framework nicht. Für gut testbare Anwendungen müssen die `data-testid`-Attribute daher bewusst und sinnvoll im Anwendungscode gesetzt werden.

Neben den Test-IDs sind die Komponenten auf Barrierefreiheit ausgelegt und folgen Accessible Rich Internet Applications (ARIA)-Konventionen *52°North GmbH, Arne Vogt, 2025*. Elemente lassen sich dadurch beispielsweise über ihre Rolle oder ihren sichtbaren Textinhalt finden. Playwright kann anhand von Rolle und Bezeichnung (`getByRole/getByText`) auf diese Elemente zugreifen. Damit bietet Trails zwei robuste Wege, ein Element zu adressieren.

2.4 WebGIS-Anwendungen und Testkomplexität

WebGIS-Anwendungen stellen die Testautomatisierung vor Herausforderungen, die generische Webanwendungen in dieser Form nicht haben. Der Grund dafür ist, wie Karten und Geodaten im Browser dargestellt und geladen werden. Ein zentrales Problem ist die Darstellung der Karte selbst. OpenLayers zeichnet die Karte auf ein Canvas-Element, also als Pixelgrafik und nicht als einzelne DOM-Elemente *OpenLayers (Institution), 2026*. Ein Locator kann zwar das Canvas-Element finden, aber nicht die Inhalte darin, z.B. ob ein bestimmter Layer gerade sichtbar ist. Was in der Karte dargestellt wird, ist also nicht direkt über das DOM zugänglich. Zu testen, ob ein Feature auf der Karte sichtbar ist, lässt sich daher nicht auf demselben Wege prüfen wie die Sichtbarkeit einer Schaltfläche.

Hinzu kommt, dass Geodaten asynchron über das Netz geladen werden. Kartenkacheln, Web Map Service (WMS)-Layer, GetFeatureInfo-Abfragen oder Anfragen des Geocoders treffen mit unterschiedlicher Verzögerung ein. Ein Test muss abwarten, bis die Daten geladen und gerendert wurden, bevor getestet werden kann. Das in Abschnitt 2.2.2 beschriebene automatische Warten von Playwright hilft hier nur bedingt, da es sich auf die Bedienbarkeit von DOM-Elementen bezieht. Ob Kartenkacheln vollständig dargestellt werden, lässt sich darüber nicht erkennen, sodass auf andere Methoden wie das Abwarten einer Netzwerkantwort zurückgegriffen werden muss.

Die Oberfläche einer WebGIS-Anwendung ist außerdem stark vom aktuellen Zustand abhängig. Ein Layer kann sichtbar oder unsichtbar sein, die Legende bezieht sich auf sichtbare Layer und eine Informationsanzeige erscheint erst, nachdem ein Feature in der Karte angeklickt wurde. Auch hier ist das Verhalten von Interaktionen wichtig. Die Informationsanzeige bei Klick auf ein Feature kann zum Beispiel deaktiviert sein, wenn gerade eine Messfunktion zum Messen einer Linie aktiv ist. Dieselbe Interaktion kann je

nach Zustand zu einem anderen Ergebnis führen, weshalb die Vorbedingungen eines Tests genau definiert sein müssen.

Außerdem sind viele Interaktionen abhängig von Koordinaten. Angenommen der Nutzer klickt auf eine Position, auf eine Markierung oder zeichnet durch mehrere Klicks eine Linie. Der Test muss dafür auf Pixelkoordinaten im Canvas klicken statt auf ein bekanntes Element. Das Ergebnis hängt dann von Kartenausschnitt, Zoomstufe und Projektion ab. Ein Klick auf eine beliebige Position ist dabei etwas anderes als das gezielte Treffen eines bestimmten Features, das an einer bekannten Stelle liegt.

3 Stand der Forschung

3.1 LLM-gestützte Testgenerierung

Die automatisierte Testgenerierung mit LLMs ist ein Teilgebiet des in Kapitel 2.1.2 beschriebenen Einsatzes von LLMs im Software Engineering (*Hou et al., 2024*). *Celik & Mahmoud (2025)* fassen die bestehenden Ansätze in einer Literaturübersicht zusammen und unterscheiden zwischen vier methodischen Kategorien: prompt-basierte Verfahren, feedback-getriebene Verfahren, Fine-Tuning und hybride Kombinationen. Der Großteil der Literatur befasst sich mit der Generierung von Unit-Tests aus dem Quellcode der jeweiligen Anwendung (ebd.). Diese Arbeit verfolgt einen anderen Ansatz: Der Test soll aus einer fachlichen Anforderung entstehen, ohne dass der Quellcode der Anwendung als direkte Eingabe dient.

Dass natürlichsprachliche Beschreibungen als alleinige fachliche Eingabe ausreichen können, zeigen *Plein et al. (2023)* in einer Machbarkeitsstudie. Sie untersuchen, ob LLMs aus natürlichsprachlichen Bug Reports ausführbare Testfälle erzeugen können. Für einen relevanten Teil der Bug Reports gelingt die Generierung fehlerproduzierender Tests. Zugleich treten typische Schwächen auf, etwa unvollständige oder nicht kompilierbare Ausgaben (ebd.). Die Studie bleibt allerdings auf der Unit-Test-Ebene und ohne Bezug zu einer Benutzeroberfläche, weshalb sich die Ergebnisse nicht direkt auf E2E-Tests übertragen lassen.

Für den E2E-Testbereich liegt eine Fallstudie von *Ferreira et al. (2025)* vor. Bei dem dort vorgestellten Ansatz erzeugt ein LLM zunächst aus User Stories Testszenarien im Gherkin-Format und generiert daraus anschließend ausführbare Tests für das Testframework Cypress. Als Kontext erhält das Modell neben der User Story auch den HTML-Code der betroffenen Seiten, der UI-Kontext wird also in fester Form mitgeliefert. Der Großteil der generierten Szenarien (95 %) und Skripte (92 %) wurde als brauchbar bewertet; von den Skripten waren 60 % direkt einsetzbar, weitere 8 % nach kleineren Korrekturen (ebd.). Zudem zeigte sich, dass die semantische Relevanz der generierten Tests von 60 % auf 92 % stieg, wenn der bereitgestellte Kontext nachträglich mitgeliefert wurde. Fehlender Kontext war zugleich die häufigste Fehlerursache. Dieser Zusammenhang zwischen Kontextqualität und Testqualität wurde jedoch nur beobachtet und nicht als kontrollierte Variable systematisch gemessen. Außerdem ist zu beachten, dass die Zielanwendungen generische Webanwendungen mit formular- und navigationsbasierten Interaktionen sind und keine karten- oder canvas-basierten Oberflächen beinhalten.

Einen anderen Weg zur Kontextgewinnung wählen *Alian et al. (2024)* mit AutoE2E. Der Ansatz identifiziert Features einer Webanwendung automatisch und leitet daraus vollständige E2E-Tests ab. Der Kontext stammt hier nicht aus einer Anforderungsbeschreibung, sondern wird durch Erkundung der Anwendung selbst gewonnen, indem der UI-Zustand systematisch gecrawlt wird. AutoE2E erreicht eine höhere Feature-Abdeckung als bisherige Verfahren und agentenbasierte Baselines und bildet damit den aktuellen Stand der LLM-gestützten E2E-Testgenerierung für Webanwendungen ab (ebd.). Von dem Ansatz dieser Arbeit unterscheidet sich AutoE2E in zwei Punkten: Die Tests entstehen aus dem beobachteten Anwendungszustand statt aus fachlichen Use Cases, und die Form des UI-Kontexts, den das LLM erhält, ist durch die Architektur fest vorgegeben und ändert sich nicht.

Die betrachteten Arbeiten belegen gemeinsam, dass die Generierung ausführbarer Tests für Webanwendungen aus natürlichsprachlichen Anforderungen machbar ist (*Plein et al., 2023; Ferreira et al., 2025; Alian et al., 2024*). Alle Ansätze nutzen dabei eine Form von Anwendungs- oder UI-Kontext, sei es der HTML-Code der Seiten oder der gecrawlte Anwendungszustand. Allerdings untersucht keine der Arbeiten, wie sich Form und Umfang des bereitgestellten UI-Kontexts auf die Qualität der generierten Tests auswirken.

3.2 Forschungslücke und Einordnung dieser Arbeit

Für WebGIS-Anwendungen mit canvas-basiertem Kartenrendering existieren nach eigener Recherche keine wissenschaftlichen Arbeiten zur LLM-gestützten Testgenerierung. Die in Kapitel 3.1 betrachteten Ansätze setzen durchgängig DOM-basierte Oberflächen voraus, deren Elemente über Selektoren ansprechbar sind. Diese Annahme trifft aber für einige Bestandteile von WebGIS-Anwendungen nicht zu. Auch für das Framework Open Pioneer Trails liegen keine wissenschaftlichen Untersuchungen vor. Ob und wie sich die Ergebnisse auf diesen Anwendungsfall übertragen lassen, ist damit offen.

Die zweite Lücke betrifft den UI-Kontext selbst. Dass der bereitgestellte UI-Kontext Einfluss auf die Ausgabequalität eines LLM hat, ist durch das In-Context Learning (vgl. Kapitel 2.1.1) zu erklären. In der Literatur zur E2E-Testgenerierung fehlt eine Untersuchung, die diesen Einfluss als einzige unabhängige Variable untersucht und alle übrigen Faktoren konstant hält.

Diese Arbeit setzt an beiden Lücken an. Sie schließt an den Workflow von der Anforderung zum Test an (*Plein et al., 2023; Ferreira et al., 2025*) und grenzt sich damit von editorbasierten Werkzeugen ab, die aus dem umgebenden Quellcode

generieren. Ihr Beitrag lässt sich in drei Punkte zusammenfassen. Erstens liefert sie erste Untersuchungen zu LLM-gestützter E2E-Testgenerierung für WebGIS-Anwendungen, einschließlich Interaktionen mit der Karte. Zweitens wird der Einfluss des UI-Kontexts als einzige unabhängige Variable untersucht, und drittens wird der für DOM-basierte Tests nicht sichtbare Kartenzustand prüfbar gemacht.

4 Methodik und Forschungskonzept

4.1 Überblick und Forschungsdesign

Im Folgenden werden die Methodik und das Forschungskonzept zur Beantwortung der Forschungsfrage dargestellt, welchen Einfluss der UI-Kontext auf die Qualität LLM-generierter Playwright-E2E-Tests für Open Pioneer Trails (OPT)-basierte WebGIS-Anwendungen hat. Die in der Einleitung formulierten Teilfragen werden dabei in das Untersuchungsdesign einbezogen.

Der UI-Kontext ist die einzige unabhängige Variable. Alle anderen Faktoren wie Use Cases, LLM und dessen Konfiguration, OPT-Playwright-Skill sowie die manuell erstellten Referenztests werden konstant gehalten. Nur unter diesen Bedingungen sind die Qualitätsunterschiede der generierten Tests dem Kontext zurechenbar und werden nicht durch andere Einflüsse verfälscht. Dieses Vorgehen knüpft an das in Kapitel 2.1.1 beschriebene In-Context Learning an, bei dem der Kontext im Prompt die Ausgabe hauptsächlich bestimmt.

Von der unabhängigen Variable abzugrenzen sind die technischen Eigenschaften, die die Anwendung selbst zur Überprüfbarkeit bereitstellt. Während der UI-Kontext beschreibt, was dem LLM im Prompt mitgeteilt wird, bezeichnen Maßnahmen wie die Vergabe von data-testid-Attributen und der Zugriff auf das Kartenmodell über `__openPioneerMap` (siehe Kapitel 5), was die Anwendung unabhängig vom Prompt für Tests bereitstellt. Diese Möglichkeiten bestehen bereits ab Stufe 1 und bleiben über alle vier Stufen unverändert. Variabel ist nur, ob und in welcher Form dem LLM im Prompt mitgeteilt wird, dass sie existieren. Die Bereitstellung dieser technischen Möglichkeiten ist damit keine eigene Stufe des Experiments, sondern eine über alle Stufen geltende Voraussetzung, ohne die der UI-Kontext nicht die einzige unabhängige Variable wäre.

Generiert werden die Tests aus den fachlichen Anforderungen des jeweiligen Use Case und nicht aus dem Quellcode der Anwendung. Die Struktur des Prompts wird dabei über alle vier Stufen konstant gehalten. Es ändert sich nur der enthaltene UI-Kontext. Verschiedene Prompting-Strategien sind nicht Teil dieser Untersuchung. Da die Generierung nicht deterministisch erfolgt, wird jeder Use Case pro Stufe mehrfach generiert (N=50). Verglichen werden dadurch Verteilungen der Testqualität je nach Stufe, nicht einzelne Ausgaben.

Als kontrollierte Konstante im Prompt wird ein OPT-Playwright-Skill verwendet. Als OPT-Playwright-Skill wird im Rahmen dieser Arbeit eine feste, appunabhängige Fähigkeit

bezeichnet, die dem LLM bei jeder Generierung als Teil des Prompts mitgegeben wird. Sie legt allgemeine Umgangsweisen für den erzeugten Testcode fest, darunter die Verwendung von Playwright mit TypeScript, den Umgang mit `async`, `await`, OPT-spezifische Besonderheiten sowie das erwartete Ausgabeformat. App-spezifische Selektoren enthält der Skill bewusst nicht, um eine Vermischung mit dem UI-Kontext zu vermeiden. Er wird unabhängig davon über alle vier Stufen unverändert angewendet. Die genaue Ausgestaltung des Skills wird in Kapitel 4.4 beschrieben.

Die vier Stufen bilden einen Verlauf von minimalem zu maximalem UI-Kontext. Stufe 1 gibt dem LLM keine Informationen über die Oberfläche, während die höheren Stufen den Kontext schrittweise erweitern und strukturieren. Abbildung 3 stellt die vier Ausprägungen mit steigendem Informationsgehalt dar.

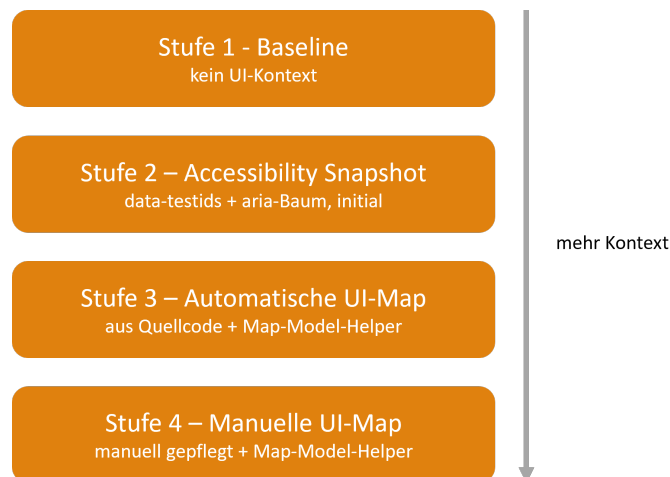


Abbildung 3: Die vier Ausprägungen der Variable UI-Kontext mit steigendem Informationsgehalt

Wie ein einzelner Generierungsvorgang abläuft, zeigt Abbildung 4. Use Case, OPT-Playwright-Skill und der UI-Kontext der jeweiligen Stufe bilden zusammen den Prompt für das LLM. Die daraus erzeugten Playwright-Tests durchlaufen ein zweiphasiges Evaluationsverfahren. In der ersten Phase werden sie gegen die laufende Demo-Anwendung ausgeführt und ihr Ausführungsergebnis klassifiziert. In der zweiten Phase bewertet ein zweites LLM die Testqualität anhand fester Kriterien, wobei die manuellen Referenztests aus Kapitel 5.3 als Vergleichsmaßstab dienen. Beide Phasen werden in Kapitel 4.5 beschrieben.

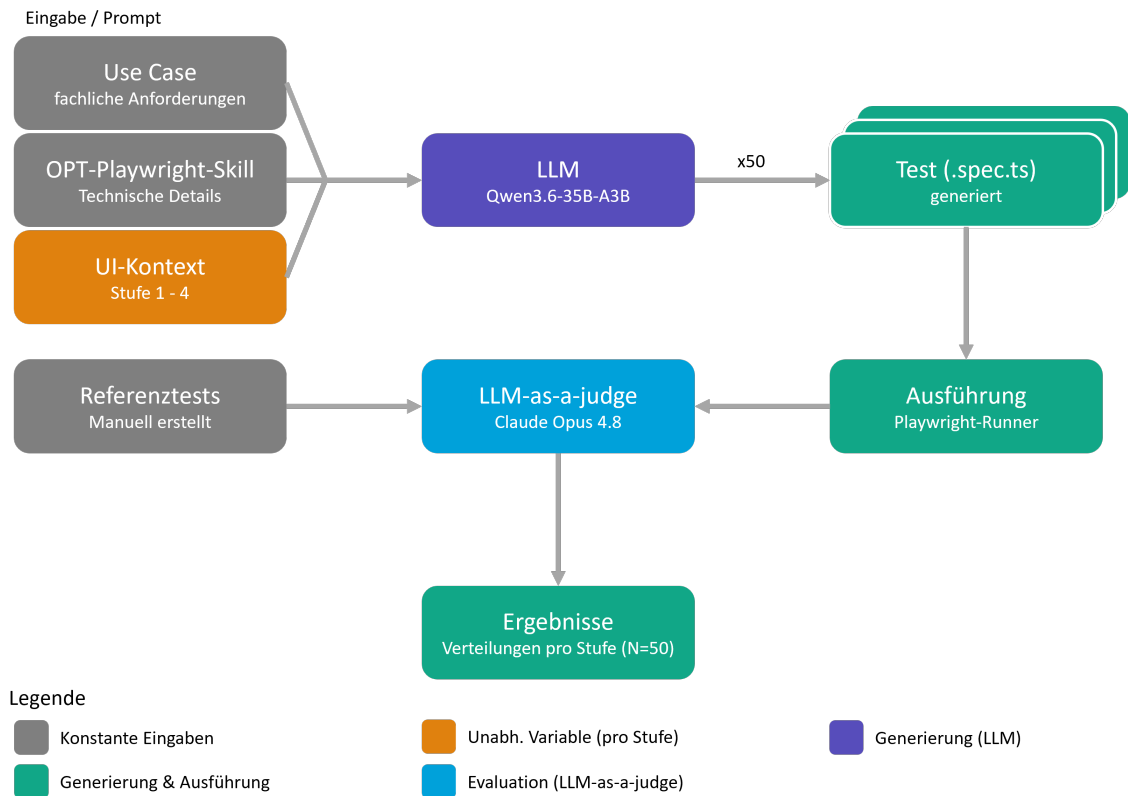


Abbildung 4: Generierungs- und Evaluationspipeline. Der UI-Kontext variiert als unabhängige Variable (Stufen 1-4), alle übrigen Eingaben sind konstant

4.2 Use Cases: Definition und Format

Die Use Cases sind die fachliche Eingabe, aus der die Tests generiert werden. Sie sind über alle vier Stufen identisch und gehören damit zu den konstant gehaltenen Faktoren.

Jeder Use Case beschreibt eine Nutzerinteraktion mit der WebGIS-Anwendung auf fachlicher Ebene aus Sicht des Anwenders. Die Schritte enthalten bewusst keine Selektoren oder andere technischen Details der Oberfläche, damit das Wissen allein aus dem UI-Kontext der jeweiligen Stufe kommt. Zudem ist es nicht sinnvoll, in einer Nutzerinteraktion technische Details der Anwendung festzuhalten, da der Anwender diese ebenfalls nicht kennt.

Die Use Cases werden im Markdown-Format erfasst. So sind sie menschenlesbar, für LLMs einfach zu verarbeiten und folgen einer einheitlichen Struktur, die für alle Stufen gleich verwendet wird. Jeder Use Case besteht aus Informationen zu `id`, `title`, `description`, `preconditions`, `steps`, `expected results` und `complexity`. Titel und Beschreibung geben an, welche Interaktion der Use Case abbildet. Die Vorbedingungen halten fest, was vor der Ausführung erfüllt sein muss. Die Schritte

beschreiben die durchzuführende Interaktion des Anwenders und die erwarteten Ergebnisse legen fest, welcher Zustand danach eintreten soll.

Die Eigenschaft `complexity` gruppiert die Use Cases in drei Schwierigkeitsstufen: `easy`, `medium`, `hard`. Die Einstufung basiert nicht allein auf der Anzahl der Schritte, sondern vor allem auf der Art und Tiefe der Interaktionen. Einfache Use Cases prüfen einzelne Schaltflächen oder Sichtbarkeiten, komplexere fordern hingegen asynchrone Prozesse, Karteninteraktionen oder das Zusammenspiel mehrerer Komponenten. Die Abstufung stellt sicher, dass bei der Untersuchung verschiedene Komplexitäten typischer WebGIS-Interaktionen abgedeckt und nicht nur einfache Fälle betrachtet werden. Ausführliche Informationen zum Inhalt der Use Cases gibt es in Kapitel 5.2.

4.3 Versuchsaufbau: Die vier Kontextstufen

4.3.1 Stufe 1: Baseline (kein UI-Kontext)

In Stufe 1 bleibt der UI-Kontext-Block des Prompts leer. Das LLM erhält nur den Use Case und den OPT-Playwright-Skill, sonst nichts. Selektoren und Verifikationsstrategien muss es aus allgemeinem Wissen über Webanwendungen ableiten, z. B. durch das Raten plausibler Beschriftungen oder ARIA-Rollen. Diese Stufe ist der Referenzpunkt des Experiments und soll zeigen, was ohne Informationen über die konkrete Oberfläche erreichbar ist. Die in Kapitel 4.1 beschriebenen Testbarkeitsmaßnahmen der Anwendungen existieren auch hier schon, dem LLM wird ihre Existenz aber nicht mitgeteilt.

4.3.2 Stufe 2: Automatischer Accessibility-Snapshot

Stufe 2 erhält den UI-Kontext automatisch aus der laufenden Anwendung. Ein Headless-Browser lädt die Demo-App und erfasst alle vergebenen `data-testid`-Werte und den Accessibility-Snapshot von Playwright, der Rollen, Namen und Zustände der Elemente als Baumstruktur ausgibt. Der Snapshot wird einmal vor dem Generierungslauf erfasst und für alle Use Cases identisch verwendet.

Der Ansatz benötigt keinen manuellen Pflegeaufwand und wäre damit in der Praxis ein günstiges Szenario. Erfasst wird allerdings nur der initiale Seitenzustand. Elemente, die erst nach Interaktionen erscheinen, fehlen im Snapshot. Auch das Kartenmodell taucht in diesem Kontext nicht auf, da `__openPioneerMap` ein globales JavaScript-Objekt ist und weder im DOM noch im Accessibility-Baum erscheint.

4.3.3 Stufe 3: Automatisch generierte UI-Map

Den gesamten Quellcode der Anwendung in den Prompt zu übernehmen ist wegen der begrenzten Kontextlänge des LLM nicht praktikabel. Stufe 3 verwendet deshalb eine automatisch aus dem Quellcode generierte UI-Map. Die regexbasierte Extraktion durchsucht alle `.ts`- und `.tsx`-Dateien der Anwendung und erfasst `data-testid`s, Beziehungen zwischen Komponenten, UI-Zustände, Sichtbarkeitsbedingungen und Interaktionsabhängigkeiten. Informationen zur Sichtbarkeit der Layer und Hintergrundkarten werden zusätzlich aus der Service-Konfiguration (`services.ts`) gelesen. Das Ergebnis wird dann als Markdown-Tabelle gespeichert und in den Prompt übernommen. Zusätzlich enthält der Prompt erstmals die Datei `map-model-helpers.ts`. Deren Funktionen helfen beim Zugriff auf das Kartenmodell und geben dem LLM damit ein Mittel, den Kartenzustand in Assertions zu prüfen.

Der Kontext in dieser Stufe stammt aus dem Quellcode statt aus der laufenden Seite. Er enthält deshalb auch Elemente, die im initialen Zustand nicht sichtbar sind. Der Ansatz bleibt automatisiert, setzt aber Zugriff auf den Quellcode voraus, und seine Qualität hängt davon ab, wie zuverlässig die regexbasierte Extraktion die relevanten Informationen findet.

4.3.4 Stufe 4: Manuell erstellte UI-Map

Stufe 4 folgt dem Aufbau von Stufe 3, ersetzt die generierte UI-Map allerdings durch eine manuell erstellte Fassung. Sie wurde mit vollständiger Kenntnis der Anwendung erfasst und enthält alle Informationen zu Selektoren sowie Hinweise zu Interaktionen und Zuständen einzelner Komponenten. Anders als die Tabelle aus Stufe 3 bildet sie die Komponenten hierarchisch ab und modelliert dynamische Sichtbarkeiten mit expliziten Bedingungen. Damit diese Beziehungen effizient dargestellt werden können, werden die Informationen im JavaScript Object Notation (JSON)-Format gespeichert. Die Map-Model-Helfer sind identisch zu Stufe 3.

Diese Stufe bildet die theoretische Obergrenze des untersuchten Ansatzes ab. Sie soll zeigen, welche Testqualität erreichbar ist, wenn der Kontext vollständig und geprüft vorliegt. Dem steht allerdings der Aufwand für die Erstellung und Pflege gegenüber. Jede Änderung an der Oberfläche muss in der UI-Map nachgezogen werden, sonst enthält sie veraltete Selektoren oder falsche Informationen zu Sichtbarkeiten und Interaktionen. In der Evaluation wird sich zeigen, ob der Qualitätsgewinn diesen Aufwand im Vergleich zu den Testergebnissen der anderen Stufen rechtfertigt.

4.3.5 Bonus-Stufe 5: Self-Improvement-Loop

Ergänzend zum Vergleich der vier Stufen wird in der Bonus-Stufe 5 eine Selbstverbesserungsschleife untersucht. Sie dient als Erweiterung der anderen Stufen und verfolgt einen etwas anderen Ansatz. Statt den statischen UI-Kontext im Prompt weiter zu verbessern, erhält das LLM Rückmeldung aus der Ausführung seines eigenen Tests, um diesen weiter zu optimieren.

Als Ausgangspunkt dient der Kontext aus dem automatischen Snapshot des initialen Seitenzustands aus Stufe 2, ergänzt um einen Screenshot der Anwendung und den Map-Model-Helfer aus Stufe 3 und 4. Den Ablauf der Schleife zeigt Abbildung 5. Zuerst wird ähnlich wie bei Stufe 2 aus dem initialen Kontext ein Test generiert und mit Playwright direkt gegen die laufende Anwendung ausgeführt. Besteht er, endet die Schleife. Schlägt er fehl, wird der nächste Prompt um den fehlgeschlagenen Testcode, die Playwright-Fehlermeldung und den Failure-Snapshot inklusive Screenshot ergänzt. Das Modell erhält damit die Anweisung, den vorhandenen Test zu korrigieren, statt einen neuen zu erzeugen. Nach spätestens 10 Iterationen bricht die Schleife ab.

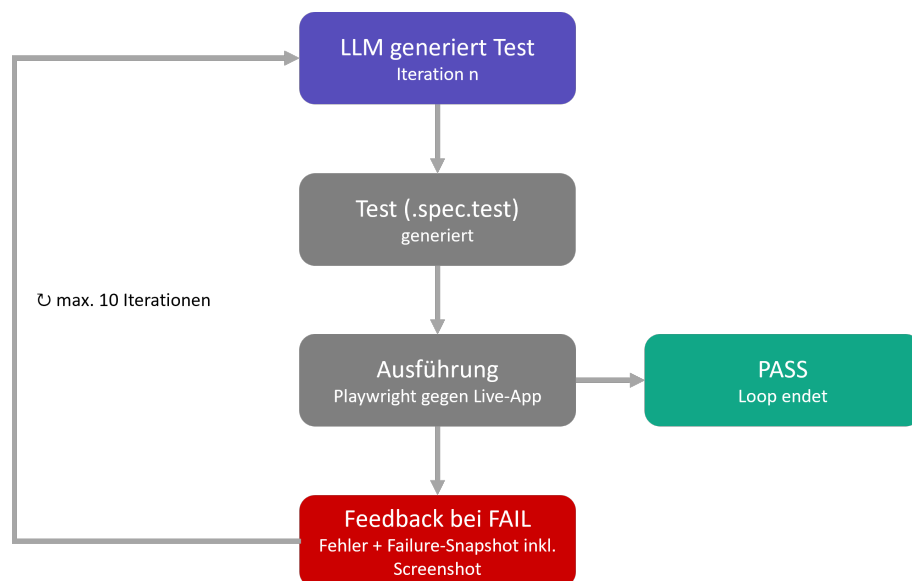


Abbildung 5: Self-Improvement-Loop der Bonus-Stufe 5: Fehlermeldung und Failure-Snapshot inkl. Screenshot fließen in den nächsten Generierungsprompt zurück (max. 10 Iterationen)

Der Failure-Snapshot hält den Zustand der Anwendung zum Zeitpunkt des Fehlschlags fest und besteht wie in Stufe 2 aus Accessibility-Snapshot und data-testids. Der Unterschied liegt im Zeitpunkt der Aufnahme. Hier wird nicht der initiale Zustand, sondern die Seite nach den bereits ausgeführten Testschritten erfasst. Das LLM sieht damit, wie die Oberfläche aussah, als der Test scheiterte. Zusätzlich erhält das LLM in der initialen und jeder weiteren

Iteration einen Screenshot der Anwendung als Bildeingabe. Dadurch bekommt es auch erstmals visuelle Informationen über den gerenderten Kartenzustand und sieht optisch, welche Elemente sichtbar sind. Als Pass-Kriterium dient der Screenshot nicht. Die Schleife endet nur über das Ausführungsergebnis des Tests. Schlägt der Test beispielsweise fehl, obwohl der Layer im Screenshot sichtbar ist, liegt der Fehler in der Prüflogik des Tests, und die Iteration wird fortgesetzt. Der Screenshot ist nur ein Hilfsmittel für die Korrektur.

Jede Iteration verwendet einen eigenständigen Prompt ohne Konversationshistorie. Alle Informationen aus dem vorherigen Durchlauf werden ausschließlich über den vorherigen Testcode, die Fehlermeldung und den Snapshot übergeben. Für die Aufnahme des Failure-Snapshots wird eine Playwright-Fixture benötigt, die in einer Ausführungskopie des Tests eingebunden wird. Die vom LLM generierte Originaldatei bleibt dabei unverändert und ist Grundlage der Bewertung.

4.3.6 Erwartungen

4.4 LLM-Konfiguration (Modell, Temperatur, Prompt-Struktur)

Für alle Generierungen wird das Qwen3.6-35B-A3B als LLM verwendet. Es ist ein offenes Sprachmodell des Qwen-Teams von Alibaba und wird in der FP8-quantisierten Variante eingesetzt. Das Modell verwendet eine Sparse-Mixture-of-Experts-Architektur mit 35 Milliarden Parametern insgesamt, von denen pro Token nur etwa 3 Milliarden aktiv sind. Zudem ist es multimodal, verarbeitet also neben Text auch Bilder. Insgesamt unterstützt es eine Kontextlänge von 262.144 Token (*Hugging Face*, 2026; *Qwen Team, Alibaba*, 2026).

Das Modell wird auf einem Spark im Netzwerk der con terra GmbH betrieben und dort über LM Studio bereitgestellt. Die Generierungsskripte greifen über die OpenAI-kompatible Representational State Transfer (REST)-API von LM Studio darauf zu.

Die Temperatur ist auf 0.6 gesetzt, da dieser Wert vom Hersteller für präzise Coding-Aufgaben empfohlen wird (*Hugging Face*, 2026). Qwen3.6 arbeitet standardmäßig im Thinking-Mode, der auch hier verwendet wird. Damit Reasoning und Testcode zusammen ausreichend Platz haben, beträgt die maximale Ausgabelänge 64k Token. Diese Konfiguration ist über alle Stufen und Use Cases identisch. Da die Generierung bei einer Temperatur über null nicht deterministisch ist, wird der vollständige Generierungsdurchlauf pro Stufe 50-mal wiederholt. Jeder Durchlauf erzeugt alle zehn Use Cases in einem eigenen Verzeichnis, sodass pro Stufe 500 Tests generiert werden.

Der Prompt besteht aus zwei Teilen. Der User-Prompt enthält die Anweisung, aus dem Use Case mit dem jeweiligen UI-Kontext einen Playwright-E2E-Test zu generieren, den Use Case selbst, die Uniform Resource Locator (URL) der Anwendung und den UI-Kontext-Block der jeweiligen Stufe. Zwischen den Stufen variiert ausschließlich der UI-Kontextblock. Im System-Prompt ist der OPT-Playwright-Skill enthalten.

Der Skill legt fest, wie der erzeugte Testcode auszusehen hat. Er schreibt die Verwendung von Playwright mit TypeScript vor, verlangt die durchgängige Nutzung von `async` und `await`, priorisiert `getByTestId` gegenüber anderen Locator-Strategien und definiert das Ausgabeformat. Die Ausgabe soll eine einzelne, vollständige Testdatei ohne erläuternden Text sein. Außerdem enthält der Skill weitere technische Vorgaben, etwa zum Umgang mit Lokatoren, Assertions, Wartebedingungen und Chakra-UI-Elementen. Der Skill ist so aufgebaut, dass er auch für andere WebGIS-Anwendungen wiederverwendet werden kann, die mit Open Pioneer Trails erstellt wurden. Informationen über App-spezifische Selektoren oder Elemente enthält er daher bewusst nicht.

4.5 Evaluationsverfahren

Die Evaluation der generierten Tests erfolgt in zwei Phasen. Zuerst werden alle Tests gegen die laufende Demo-Anwendung ausgeführt und anschließend mit einem LLM-as-a-Judge-Verfahren semantisch bewertet. Beide Phasen sollen sich ergänzen. Ein erfolgreich ausgeführter Test prüft nicht zwangsläufig das im Use Case vorgegebene Verhalten, während umgekehrt ein fachlich korrekter Test bereits an einem einzelnen fehlerhaften Selektor scheitern kann. Eine manuelle semantische Bewertung ist aufgrund der Menge von 2.500 Testdateien (50 Generierungen $\times$ 10 Use Cases $\times$ 5 Stufen) ausgeschlossen. Als Bewertungseinheit dient jede einzelne Testdatei; ausgewertet werden die Ergebnisse anschließend je Stufe als Verteilung, nicht als Einzelurteile.

4.5.1 Phase 1: Deterministische Ausführung

In Phase 1 werden alle generierten Testdateien mit Playwright gegen die lokal laufende Demo-Anwendung ausgeführt. Die Ausführung erfolgt nicht manuell, sondern über ein Skript: Nach jedem Testlauf liest es das Ergebnis über den JSON-Reporter von Playwright ein und ordnet es einer von sechs Kategorien zu. Maßgeblich ist dabei jeweils die erste Stelle, an der ein Test scheitert.

GENERATION_ERROR Der Test entsteht erst gar nicht als valider Code, da die Modellantwort abgeschnitten ist oder Wiederholungsmuster enthält. Erkannt wird dies unter anderem durch eine Klammerprüfung.

COMPILE_ERROR TypeScript- oder Importfehler verhindern die Ausführung, ohne dass die Datei als abgeschnitten erkannt wurde, etwa der Import eines nie bereitgestellten Hilfsmoduls.

INFRA_FAIL Der Test läuft, scheitert aber, bevor er eine inhaltliche Prüfung erreicht, etwa weil ein Selektor sein Element nicht findet oder mehrdeutig ist. Die `expect`-Aussage wird nie erreicht.

ASSERTION_FAIL Der Test erreicht seine Prüfung und die Selektoren lösen ihre Elemente auf, aber die Erwartung trifft nicht zu: Das Ergebnis oder der Zustand ist ein anderer als angenommen.

TIMEOUT Ein äußerer Prozess-Guard bricht den Lauf ab, weil der Playwright-Test selbst nicht terminiert.

PASS Der Test läuft vollständig und erfolgreich durch.

Die Zuordnung erfolgt regelbasiert über Muster in den Fehlermeldungen des Playwright-Reports. Fälle, die keinem Muster eindeutig entsprechen, werden für eine manuelle Nachklassifikation markiert.

Für Stufe 5 entfällt eine separate Ausführung, da die generierten Tests schon in jeder Iteration des Loops gegen die Anwendung ausgeführt werden. Das Ausführungsergebnis der letzten Iteration wird im Loop-Protokoll gespeichert, sodass ein Abbildungsskript mit identischer Klassifikationslogik die Tests aus dieser Stufe in dasselbe Kategorieschema überführt.

4.5.2 Phase 2: LLM-as-a-Judge

Die semantische Bewertung übernimmt ein Sprachmodell, das die generierten Tests anhand vorgegebener Bewertungskriterien beurteilt. *Zheng et al. (2023)* haben dieses als LLM-as-a-Judge bezeichnete Verfahren untersucht und für leistungsfähige Modelle eine Übereinstimmung mit menschlichen Bewertungen von über 80 % nachgewiesen. Sie unterscheiden drei Bewertungstypen, die einzeln oder kombiniert eingesetzt werden können. Beim „Pairwise Comparison“ werden zwei Antworten auf dieselbe Frage gegeneinander verglichen, beim „Single Answer Grading“ wird eine einzelne Antwort auf einer absoluten

Skala eingeordnet, und das „Reference-guided Grading“ zieht dafür zusätzlich eine vorgelegte Musterlösung heran. Diese Arbeit kombiniert die beiden letztgenannten Typen: Jede Testdatei wird einzeln gegen eine Skala mit definierten Stufenbeschreibungen bewertet, wobei der manuell erstellte Referenztest als Musterlösung dient. Als mögliche Verzerrung der Ergebnisse nennen sie den Self-Enhancement-Bias, also die Bevorzugung selbst erzeugter Ausgaben eines Modells (Zheng et al., 2023). Ob dieser Effekt wirklich auftritt, lässt ihre Untersuchung offen. Als Vorsichtsmaßnahme stammen Generierungs- und Bewertungsmodell in dieser Arbeit daher aus verschiedenen Modellfamilien.

Als Judge-Modell wird Opus 5 von Anthropic verwendet, das am 24. Juli 2026 veröffentlicht wurde. Die Bewertung läuft agentisch über Claude Code direkt im Repository. Das Modell liest die generierten Testdateien, die Use Cases, die Referenztests und die Ergebnisdatei aus Phase 1 ein und schreibt die Bewertungen fortlaufend in die Ausgabedateien.

Der Bewertungsprompt (Anhang Anhang 2) ist bis auf die Verzeichnispfade über alle Stufen identisch. Die Stufen werden getrennt bewertet, und jede Testdatei wird ausschließlich an der Skala der Stufenbeschreibung gemessen, nicht im Vergleich zu anderen Dateien oder Stufen. Bewertet wird blockweise je Use Case über alle 50 Generierungen hinweg. Der Referenztest gilt als Maßstab für korrekte Selektoren, kartenspezifische Interaktionen und vollständige Abdeckung, nicht als Vorlage für die Syntax. Dateien, die in Phase 1 als `GENERATION_ERROR` klassifiziert wurden, bleiben unbewertet, da kein valider Testcode vorliegt.

Für Stufe 5 weicht der Bewertungsprompt (Anhang Anhang 2.2) ab, da die Ergebnisse dort wie schon in Phase 1 in einer etwas anderen Form vorliegen. Bewertet wird ausschließlich der Testcode der letzten Iteration. Die Anzahl der benötigten Iterationen erhält das Modell zwar über die Ergebnisdatei aus Phase 1, sie ist aber ausdrücklich kein Bewertungskriterium. Ein Test, der erst nach vier Iterationen besteht, wird genauso bewertet wie einer nach der ersten Iteration.

Bewertet wird in vier Dimensionen, jede auf einer Skala von 1 bis 4 mit ausformulierten Beschreibungen der einzelnen Skalenstufen. Die vollständige Beschreibung ist in Anhang Anhang 1 hinterlegt. Zu jeder Dimension wird zuerst eine kurze Begründung verlangt und erst danach der Punktwert vergeben. Die Dimension *coverage* erfasst, wie vollständig die Schritte und die erwarteten Ergebnisse des Use Cases abgedeckt sind, *selector* die Korrektheit der verwendeten Selektoren, *map_interaction* die Qualität der kartenspezifischen Interaktionen und *assertion*, ob die Prüfungen die Erwartungen des Use Cases tatsächlich verifizieren würden.

Die Dimension *map_interaction* ist nur auf einen Teil der Use Cases anwendbar, bei denen für das erwartete Ergebnis tatsächlich eine Karteninteraktion nötig ist. Bei Use Cases,

in denen z. B. nur die Sichtbarkeit einer Schaltfläche geprüft wird, ergibt sie keinen Sinn und erhält deshalb den Wert n/a . Bei den übrigen Use Cases fließt die Bewertung der Karteninteraktion dagegen in die Auswertung ein.

Zusätzlich wird für jede Datei das Merkmal *vacuous_pass* bestimmt. Es trifft zu, wenn ein Test in Phase 1 als `PASS` klassifiziert wurde, die Dimension *assertion* aber nur mit 1 oder 2 bewertet ist. Der Test läuft also erfolgreich durch, ohne das geforderte Verhalten zu prüfen.

Zur Kontrolle der maschinellen Bewertung wird je Stufe eine Stichprobe von 10 Testdateien manuell überprüft, insgesamt also 50 Dateien. Die Stichprobe ist nach Use Cases geschichtet: Aus jedem Use Case wird pro Stufe genau eine Generierung zufällig gezogen. Geprüft wird, ob die vergebenen Punktwerte den Skalenbeschreibungen der Rubrik entsprechen. Dazu werden der generierte Testcode, die Punktwerte und die zugehörigen Begründungen des Modells gemeinsam betrachtet und je Dimension erfasst, ob die Bewertung bestätigt wird oder Abweichungen vorliegen.

5 Implementierung der Evaluationsumgebung

5.1 Demo-WebGIS-Anwendung

Die Demo-WebGIS-Anwendung stellt eine realistische, aber kontrollierte Testumgebung für die Evaluation bereit. Als Single-Page-Webanwendung umgesetzt, dient sie der weltweiten Erkundung von Wetterdaten und -vorhersagen. Die Anwendung ist unter <https://nils-gb156.github.io/ba-webgis-llm-e2e/> erreichbar und kann auch lokal gestartet werden. Der Quellcode ist auf GitHub unter <https://github.com/nils-gb156/ba-webgis-llm-e2e> verfügbar.

Abbildung 6 zeigt die Oberfläche der Anwendung. Den Mittelpunkt bildet die Karte. Auf der linken Seite befinden sich die Layersteuerung und die Legende, die über die Buttons in der Werkzeugleiste unten auch optional ein- und ausgeblendet werden können. Die Layersteuerung ermöglicht das Umschalten zwischen Hintergrundkarten sowie das Ein- und Ausblenden von Layern. Die Legende darunter zeigt die Symbolerklärungen der jeweils aktiven Layer.

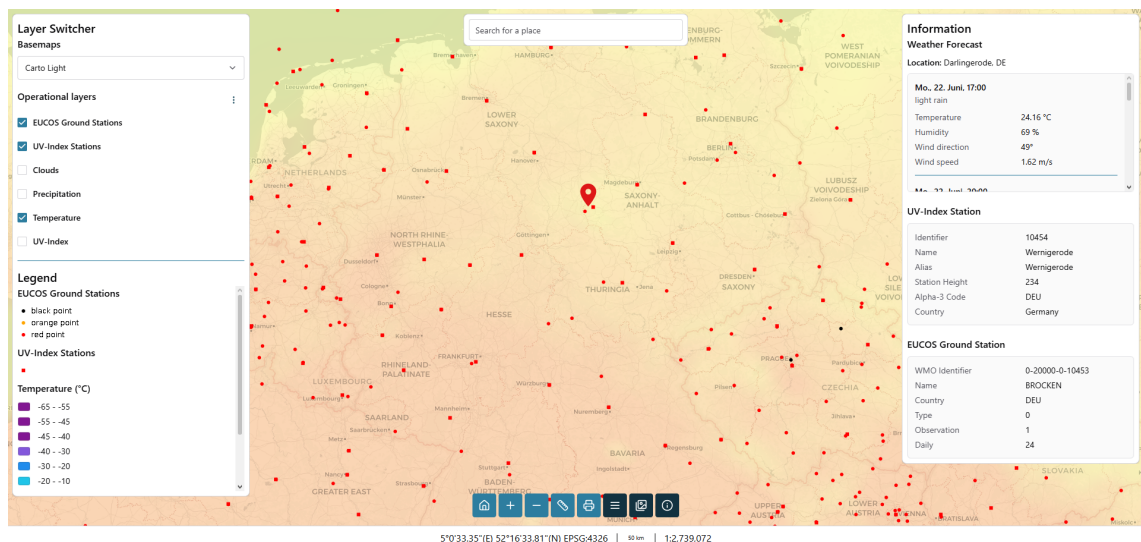


Abbildung 6: Oberfläche der Demo-WebGIS-Anwendung mit geklickter Koordinate

Die Anwendung bietet drei verschiedene Hintergrundkarten: Carto Light (Standard), Carto Dark und OpenStreetMap. Initial sichtbar sind zwei Punktlayer des Deutschen Wetterdienstes (DWD) für EUCOS-Bodenstationen und UV-Index-Stationen. Außerdem ist noch ein flächendeckender Layer der aktuellen Temperatur von OpenWeatherMap sichtbar. Über die Layersteuerung können weitere flächendeckende Layer für Niederschlag und

Wolkenbedeckung ebenfalls von OpenWeatherMap sowie ein europaweiter UV-Index-Layer des DWD eingeblendet werden.

Ein Klick auf die Karte öffnet ein Infopanel, das den Ort und die Wettervorhersage für die nächsten drei Tage im Drei-Stunden-Takt anzeigt. Die insgesamt 24 Einträge werden über die OpenWeatherMap API abgerufen und enthalten jeweils eine kurze Beschreibung des Wetters, Temperatur, Luftfeuchtigkeit sowie Windrichtung und -geschwindigkeit. Wird zusätzlich auf eine Bodenstation oder UV-Index-Station geklickt, werden die Stationsinformationen aus der WMS-GetFeatureInfo-Anfrage unterhalb der Wettervorhersage angezeigt.

Oben in der Mitte der Karte befindet sich ein Feld für eine Freitextsuche, die über die Nominatim-Geocoding-API nach Orten sucht. Bei Auswahl eines Suchergebnisses wird die Karte auf den Ort zentriert und das Infopanel mit der Wettervorhersage angezeigt.

Über die Werkzeugleiste lässt sich außerdem zur initialen Ausdehnung zurücknavigieren sowie der Kartenausschnitt vergrößern oder verkleinern. Zwei weitere Buttons rechts daneben öffnen verschiebbare Floating-Panels für Mess- und Druckfunktion. Das Messwerkzeug ermöglicht das direkte Messen von Linien und Flächen in der Karte. Mit der Druckfunktion kann der aktuelle Kartenausschnitt mit allen aktiven Layern als Portable Document Format (PDF) oder Portable Network Graphics (PNG) exportiert werden. Im Footer werden die aktuellen Kartenkoordinaten in World Geodetic System 1984 (WGS84) sowie eine Maßstabsleiste und der Maßstab angezeigt.

Die Anwendung wurde mit Open Pioneer Trails entwickelt und basiert auf OpenLayers, React, Chakra UI und TypeScript. Sie nutzt die verfügbaren OPT-Pakete für Layersteuerung, Kartennavigation, Legende, Messung, Druck und Koordinatenanzeige. Die Kartenprojektion ist European Petroleum Survey Group (EPSG):3857, die Koordinatenanzeige im Footer transformiert diese zu EPSG:4326.

5.2 Use-Case-Sammlung

Für das Experiment wurden zehn Use Cases erstellt, die typische Nutzerinteraktionen mit der Demo-WebGIS-Anwendung abbilden. Sie folgen dem in Kapitel 4.2 beschriebenen Format und verteilen sich auf drei einfache, zwei mittlere und fünf schwere Use Cases. Tabelle 1 zeigt eine Übersicht.

Die einfachen Use Cases prüfen einzelne Bedienelemente über den DOM, etwa das Ein- und Ausblenden der Layersteuerung, den Wechsel der Hintergrundkarte oder die Bedienung

Tabelle 1: Übersicht der Use Cases mit Komplexität und getesteter Kernfunktion

ID	Titel	Komplexität	Kernfunktion
1	Layer Switcher ein-/ausblenden	easy	Panel-Sichtbarkeit
2	Hintergrundkarte wechseln	easy	Basemap-Wechsel
3	Zoom-Buttons verwenden	easy	Zoom-Steuerung
4	UV-Index-Overlay aktivieren	medium	Layer-Rendering
5	Niederschlags-Overlay + Legende	medium	Layer + Legende
6	Wettervorhersage per Kartenklick	hard	Kartenklick-Abfrage
7	Stations-Info abfragen (WMS + WFS)	hard	GetFeatureInfo, GetFeature
8	Distanz messen	hard	Messwerkzeug
9	Karte als PNG exportieren	hard	Druckfunktion
10	Layer konfigurieren, Suche + Vorhersage	hard	Geocoder + Workflow

der Zoom-Buttons. Die mittleren Use Cases aktivieren zunächst ausgeblendete Layer und prüfen anschließend einen daraus resultierenden Zustand. Bei UC-4 ist es das tatsächliche Rendern der Kacheln auf der Karte und bei UC-5 die Aktualisierung der Legende.

Die schweren Use Cases hingegen erfordern Karteninteraktionen, asynchrone Prozesse oder das Zusammenspiel mehrerer Komponenten. UC-6 und UC-7 wurden trotz oberflächlicher Ähnlichkeit bewusst getrennt gehalten. UC-6 prüft die Wettervorhersage durch einen Klick auf eine beliebige Position auf der Karte, während UC-7 das gezielte Treffen einer vorgegebenen Koordinate erfordert. Hierbei muss das LLM die Koordinate zuerst in Pixelkoordinaten umrechnen, bevor der Klick simuliert werden kann. UC-8 (Streckenmessung) verlangt eine mehrstufige Zeichnung auf dem Canvas inklusive Doppelklick zum Abschluss und UC-9 prüft den PNG-Export der aktuellen Kartenansicht. Das Besondere an diesen beiden Use Cases ist, dass diese Funktionen direkt über OPT-Komponenten eingebunden sind. Bei der Erstellung der WebGIS-Anwendung besteht daher kein Einfluss darauf, wie im Test mit diesen Komponenten interagiert werden kann. Der komplexeste ist UC-10, da er Layer-Konfiguration, Geocoder-Suche, Kartennavigation und Wettervorhersage in einem einzigen Ablauf kombiniert.

Da es sich um eine Demo-Anwendung handelt, ist die Sammlung bewusst begrenzt gehalten. Reale WebGIS-Anwendungen können in der Praxis deutlich umfangreichere Interaktionen erfordern.

5.3 Manuell erstellte Referenztests

Für jeden Use Case wurde manuell ein Playwright-Test erstellt. Die Erstellung erfolgte vor der LLM-Generierung, um eine Beeinflussung zu vermeiden. Wären die Referenztests nach den generierten Tests geschrieben, bestünde die Gefahr, sich unbewusst an deren Lösungen zu orientieren. Die Referenztests haben zwei Funktionen. Zum einen dienen sie als fachlich korrekte Referenzimplementierung, die zeigt, wie ein Use Case sauber umgesetzt aussieht. Zum anderen bilden sie den Vergleichsmaßstab für die LLM-generierten Tests in der Evaluation (Kapitel 6).

Die Tests sind mit Playwright in TypeScript umgesetzt. Pro Use Case gibt es eine eigene Spec-Datei nach dem Namensschema `uc-01-*.spec.ts` bis `uc-10-*.spec.ts`. Jeder Test folgt demselben Aufbau, der die Struktur des Use Cases widerspiegelt: Zuerst wird geprüft, ob die Vorbedingungen erfüllt sind, dann werden die Schritte ausgeführt und zuletzt die erwarteten Ergebnisse überprüft. Listing 1 zeigt diesen Aufbau am Beispiel von Use Case 4.

Bevor ein Test mit der Webanwendung interagiert, muss sichergestellt sein, dass die Anwendung vollständig geladen ist. Statt einer festen Wartezeit (Timeout), die unzuverlässig ist, wird mit `page.waitForFunction(() => globalThis.__openPioneerMap !== null)` gewartet, bis das Kartenmodell verfügbar ist. Sobald das Objekt existiert, ist die Karte einsatzbereit. Der Test wartet damit auf ein deterministisches Signal und nicht auf eine geschätzte Dauer. Dieses Muster ist in jedem Test enthalten.

Listing 1: UseCase-4: UV-Index Layer einblenden und prüfen, ob es auf der Karte angezeigt wird (Playwright-Test)

```

1 import { test, expect } from "@playwright/test";
2 import { isLayerRendered } from "../../map-model-helpers";
3
4 const UVI_LAYER_TITLE = "UV-Index";
5
6 test("UC-4: activate the UV-Index overlay and verify it is rendered on the map", async
    ({ page }) => {
7     await page.goto("http://localhost:5173/ba-webgis-llm-e2e/");
8
9     const map = page.getByTestId("map-container");
10    const layerSwitcher = page.getByTestId("layer-switcher");
11    // The TOC renders each layer with a checkbox whose accessible name is the layer
12    // title. `exact` avoids also matching the "UV-Index Stations" layer.
13    const uviToggle = layerSwitcher.getByRole("checkbox", { name: UVI_LAYER_TITLE,
14        exact: true });
15    const uviLabel = layerSwitcher.getByText(UVI_LAYER_TITLE, { exact: true });
16
17    // Preconditions
18    await expect(map).toBeVisible();

```

```

17   await page.waitForFunction(
18     () => (globalThis as { __openPioneerMap?: unknown }).__openPioneerMap != null
19   );
20   await expect(layerSwitcher).toBeVisible();
21   await expect(uviToggle).not.toBeChecked();
22   expect(await isLayerRendered(page, UVI_LAYER_TITLE)).toBe(false);
23
24   // Steps
25   await uviLabel.click();
26
27   // Expected result
28   await expect(uviToggle).toBeChecked();
29   await expect.poll(() => isLayerRendered(page, UVI_LAYER_TITLE)).toBe(true);
30 });

```

Die Locator-Strategie basiert auf der in Kapitel 2 beschriebenen Hierarchie und nutzt bevorzugt `getByTestId`, dann `getByRole` und dann `getByText`. CSS-Selektoren und XPath werden vermieden, da sie an die DOM-Struktur gebunden und damit fragil sind. Die Selektoren müssen zudem bewusst präzise gewählt werden, da es sonst zu Fehlzugriffen kommen kann. Zum Beispiel wird in Use Case 4 der UV-Index Layer über `getByRole("checkbox", { name: "UV-Index", exact: true })` angesprochen. Die Option `exact` ist hier notwendig, weil sonst auch der Layer „UV-Index Stations“ mitgetroffen werden würde, was im Rahmen des Use Cases falsch wäre. Eine Ausnahme ist Use Case 9. Dort werden zwei Elemente über CSS-Klassen angesprochen, weil die betreffenden internen Elemente der Druckfunktion von Open Pioneer Trails keine eigene Test-ID besitzen.

Da der Karteninhalt auf einem Canvas gerendert wird (siehe Kapitel 2.4), gibt es dort keine DOM-Elemente. Klicks auf der Karte erfolgen daher über `page.mouse.click(x, y)` mit Pixelkoordinaten, die aus der `boundingBox()` des Kartencontainers berechnet werden. Dabei gibt es einen wichtigen Unterschied zwischen zwei Arten von Kartenklicks. Bei Use Case 6 genügt ein Klick auf eine beliebige Position, etwa die Kartenmitte. Bei Use Case 7 hingegen muss ein konkreter Punkt getroffen werden. Dafür wird die bekannte Koordinate (EPSG:3857) über die OpenLayers-Funktion `getPixelFromCoordinate()` in eine Canvas-Pixelposition umgerechnet, bevor geklickt wird. Das gezielte Treffen eines Features ist damit deutlich aufwendiger als ein beliebiger Klick in die Karte.

Der Kartenzustand, dazu gehören aktive Layer, Zoomstufe, Zentrum und Highlight, ist nicht über das DOM auslesbar. In der Datei `map-model-helpers.ts` sind Funktionen enthalten, die auf das `MapModel` zugreifen. Sie sind bewusst ausgelagert, da sie unabhängig von der Webanwendung sind und auch in anderen OPT-Anwendungen wiederverwendet werden können. Passend für die Use Cases stellt die Datei aktuell die Funktionen `getActiveBaseLayerTitle()`, `isLayerRendered()`,

`getMapZoomLevel()`, `getMapCenter()` und `getHighlightedCoordinate()` bereit. Die Funktionen müssen nur eingebunden werden und können dann direkt im Test verwendet werden.

Zum Beispiel geht es in Use Case 4 um das Aktivieren des UV-Index Layers. Dies kann nun relativ einfach auf zwei Ebenen geprüft werden. Erstens auf DOM-Ebene, dass der Toggle in der Layersteuerung den Zustand „aktiviert“ (`toBeChecked`) hat. Zweitens auf Modell-Ebene, dass der Layer auch tatsächlich gerendert wird (`isLayerRendered`). Nur weil ein Layer in der Layersteuerung aktiviert wurde, bedeutet das noch nicht, dass die Kacheln auch wirklich auf der Karte erscheinen. Die Prüfung auf Modell-Ebene nutzt `expect.poll()`, da die Kacheln asynchron über das Netz geladen werden. Ein reines `expect` ist hier nicht ausreichend, da es nur einen festen Wert prüft und nicht wiederholt nachfragt. Mit `expect.poll()` wird die Funktion wiederholt aufgerufen, bis der Layer als gerendert gilt oder ein Timeout erreicht wird. Somit wird sichergestellt, dass die Kacheln geladen sind, bevor der Test fortschreitet. Konkret sieht das dann so aus: `await expect.poll(() => isLayerRendered(page, UVI_LAYER_TITLE)).toBe(true)`.

In Use Case 7 wird zusätzlich überprüft, dass tatsächlich ein WMS-GetFeatureInfo-Request an den DWD-Dienst gesendet wird. Dazu werden mit `page.on("request", ...)` die ausgehenden Requests beobachtet.

6 Durchführung und Evaluation

6.1 Durchführung des Experiments

Die Generierungsabläufe aller Stufen fanden zwischen dem 25. und 27. Juli 2026 statt. Modellkonfiguration und Stand der Demo-Anwendung blieben über diesen Zeitraum unverändert, sodass Unterschiede zwischen den Stufen nicht auf zwischenzeitliche Änderungen an Modell oder Anwendung zurückgehen können. Verwendet wurden exakt die Parameter aus Kapitel 4.4.

Vorgesehen waren pro Stufe 500 Testdateien, also 50 Durchläufe mit jeweils zehn Use Cases. Diese Anzahl wurde in den Stufen 1, 2, 4 und 5 erreicht. Stufe 3 umfasst nur 499 Dateien, da im Durchlauf run_20 der Test zu UC-02 fehlt. Da weder die Testdatei noch der zugehörige Prompt oder die Rohantwort des Modells geschrieben wurde, lässt sich der Fehler auf den Aufruf des Modells eingrenzen, denn erst nach Rückgabe der Antwort werden alle drei Dateien angelegt. Nach einem fehlgeschlagenen Aufruf fährt das Skript mit dem nächsten Use Case fort, statt den gesamten Durchlauf abubrechen. Die genaue Ursache ist nachträglich nicht mehr bestimmbar, da die Konsolenausgabe nicht gespeichert wurde.

Ausgeführt wurden die Tests mit Playwright 1.60.0 und dem darin enthaltenen Chromium unter Node.js 26.5.0 und pnpm 11.1.2. Die Konfiguration verwendet ein einzelnes Browser-Projekt im Headless-Modus mit einem Viewport von 1920×1080 Pixeln und einem Timeout von 30 Sekunden pro Test. Alle Tests liefen nacheinander gegen die unter <http://localhost:5173/ba-webgis-llm-e2e/> laufende Demo-Anwendung.

Die semantische Bewertung nach Kapitel 4.5.2 wurde zwischen dem 31. Juli und dem 3. August 2026 mit Claude Opus 5 über Claude Code 2.1.220 durchgeführt.

Zusätzlich wurde der komplette Generierungsablauf mit GPT-5.4 statt Qwen3.6 durchgeführt. Die Ergebnisse dazu stehen im Anhang.

6.2 Ergebnisse der Ausführung (Phase 1)

Grundlage der Auswertung sind 1.999 Testdateien der Stufen 1 bis 4. Die Ergebnisse der Bonus-Stufe 5 werden wegen des abweichenden Ausführungsmodells getrennt in Kapitel 6.4 dargestellt. Jede Datei wurde einer der sechs Kategorien aus Kapitel 4.5.1 zugeordnet. Zehn Dateien entsprachen keinem Muster der regelbasierten Zuordnung,

weil sie wegen Syntaxfehlern nicht richtig ausgeführt werden konnten, und wurden daher vorläufig als `TIMEOUT` klassifiziert. Die manuelle Prüfung ergab in neun Fällen einen Syntaxfehler (`COMPILE_ERROR`) und in einem Fall eine Wiederholung ganzer Codeblöcke (`GENERATION_ERROR`).

Tabelle 2 zeigt die Ausführungsergebnisse. Die PASS-Rate steigt monoton von 20,0 % in Stufe 1 auf 38,6 % in Stufe 4. Der größte Einzelschritt liegt mit 13,7 Prozentpunkten zwischen Stufe 2 und Stufe 3, also bei der Einführung der UI-Map mit Informationen über die gesamte Web-Anwendung und der Map-Model-Helfer. Die Übergänge von Stufe 1 zu 2 (+2,0 Prozentpunkte) und von Stufe 3 zu 4 (+2,9 Prozentpunkte) verändern die PASS-Rate dagegen kaum.

Tabelle 2: Ausführungsergebnisse der Stufen 1 bis 4 nach Kategorien (Phase 1), absolut und in Prozent der Stufengrundmenge

Kategorie	Stufe 1	Stufe 2	Stufe 3	Stufe 4
PASS	100 (20,0 %)	110 (22,0 %)	178 (35,7 %)	193 (38,6 %)
INFRA_FAIL	345 (69,0 %)	211 (42,2 %)	133 (26,7 %)	145 (29,0 %)
ASSERTION_FAIL	43 (8,6 %)	166 (33,2 %)	184 (36,9 %)	154 (30,8 %)
COMPILE_ERROR	3 (0,6 %)	1 (0,2 %)	3 (0,6 %)	4 (0,8 %)
GENERATION_ERROR	9 (1,8 %)	12 (2,4 %)	1 (0,2 %)	4 (0,8 %)
TIMEOUT	0	0	0	0
<i>n</i>	500	500	499	500

Zwischen Stufe 1 und 2 sinkt der `INFRA_FAIL`-Anteil von 69,0 % auf 42,2 %, während der `ASSERTION_FAIL`-Anteil im selben Schritt von 8,6 % auf 33,2 % steigt. Der Accessibility-Snapshot verbessert demnach das Auffinden der Elemente, sodass die Tests nicht mehr am Selektor scheitern, sondern erst an der inhaltlichen Prüfung. Über die Use Cases verteilt sich dieser Effekt ungleich (Abbildung 7). Bei UC-06, UC-07 und UC-08 wandelt sich der in Stufe 1 fast vollständige `INFRA_FAIL`-Anteil bis Stufe 2 überwiegend in `ASSERTION_FAIL` um, bei UC-09 und UC-10 fällt diese Verschiebung deutlich geringer aus.

Auch die absolute PASS-Rate verteilt sich ungleich über die Use Cases. Innerhalb der Stufe 4 reicht sie von 2 % (UC-07) bis 98 % (UC-01). Use Cases mit einfachen Sichtbarkeitsprüfungen wie UC-01 und UC-05 erreichen bereits in den unteren Stufen hohe Werte, während die karteninteraktiven UC-06, UC-07 und UC-08 in allen vier Stufen unter 25 % bleiben.

Über die Stufen hinweg steigt die PASS-Rate bei den meisten Use Cases an. UC-03 ist der einzige Use Case, dessen PASS-Rate dabei durchgehend schlechter wird: 62 % in Stufe 1,

32 % in Stufe 2, 4 % in Stufe 3 und 14 % in Stufe 4. UC-04 bricht von Stufe 3 auf Stufe 4 von 66 % auf 12 % ein, obwohl die Gesamtrate in diesem Schritt steigt. UC-02 erreicht in Stufe 3 keinen einzigen PASS.

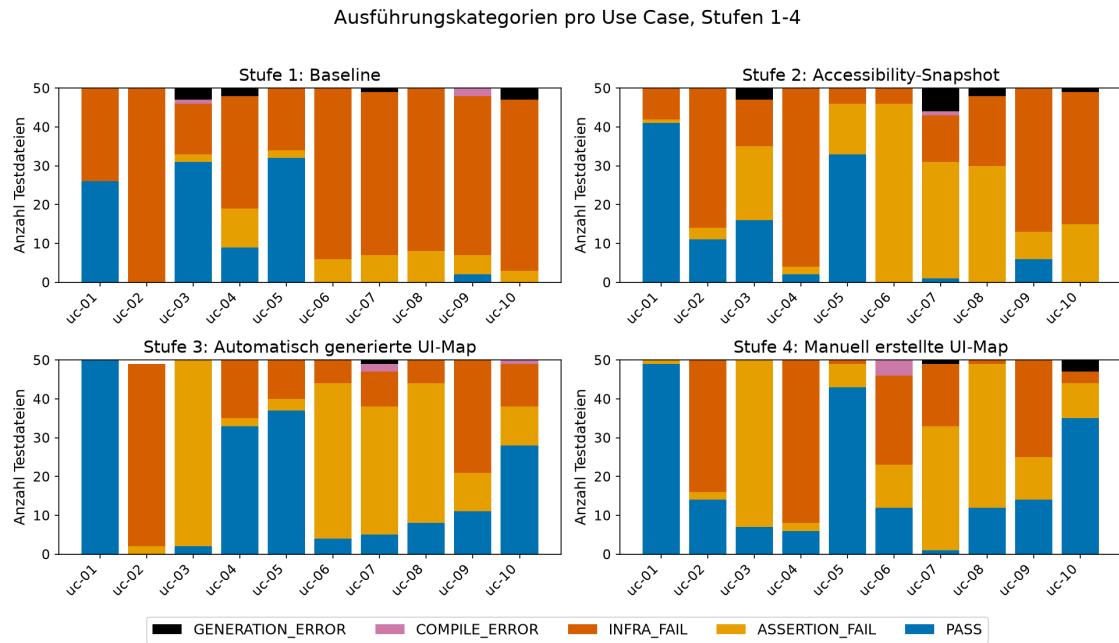


Abbildung 7: Ausführungskategorien je Use Case, Stufen 1 bis 4 (Phase 1)

Der Rückgang bei UC-03 lässt sich auf einen Fehler bei der Prüfung des Zoomverhaltens zurückführen. Ab Stufe 3 fragen die Tests den numerischen Zoomwert über `getMapZoomLevel` ab. In Stufe 3 entfallen 58,3 % der Fehlschläge dieses Use Cases auf denselben Match-Fehler („expected value must be a number or bigint“), in Stufe 4 sind es 53,5 %. Die Ursache zeigt der generierte Code in Listing 2: Die Tests weisen den Rückgabewert von `expect.poll(...)` einer Variablen zu, um ihn im nächsten Schritt als Vergleichswert zu verwenden. Da `expect.poll` als Assertion aber keinen Wert zurückgibt, erhält die Variable `undefined`, und der Vergleich scheitert. Der manuelle Referenztest in Listing 3 fragt den Zoomwert vor der Interaktion direkt über `getMapZoomLevel(page)` ab und setzt `expect.poll` nur zum Warten auf die Zustandsänderung ein, ohne dessen Rückgabewert weiterzuverwenden.

Listing 2: Fehlerhaftes Muster in den generierten Tests (gekürzt, Stufe 4, run_01)

```
1 const initialZoom = await expect.poll(() => getMapZoomLevel(page)).toBeTruthy();
2 await page.getByTestId('zoom-in-button').click();
3 const zoomedInLevel = await expect.poll(() => getMapZoomLevel(page)).toBeGreaterThan(
  initialZoom);
```

Listing 3: Korrektes Muster im manuellen Referenztest (gekürzt)

```
1 | const initialZoom = await getMapZoomLevel(page);  
2 | await page.getByTestId("zoom-in-button").click();  
3 | await expect.poll(() => getMapZoomLevel(page)).toBeGreaterThan(initialZoom as number);
```

In den Stufen 1 und 2, in denen die Map-Model-Helfer nicht im Kontext stehen, prüfen die generierten Tests für UC-03 stattdessen nur Sichtbarkeiten im DOM und laufen dadurch häufiger durch. Die hohe PASS-Rate in Stufe 1 bei diesem Use Case belegt also keine höhere Testqualität. Die Tests müssen daher zusätzlich bewertet werden, ob sie das geforderte Verhalten auch tatsächlich prüfen.

Die übrigen Kategorien machen nur einen geringen Anteil aus. `COMPILE_ERROR` und `GENERATION_ERROR` machen zusammen je Stufe zwischen 0,8 % und 2,6 % aus. Abgeschnittene oder degenerierte Modellausgaben (`GENERATION_ERROR`) treten fast nur in den Stufen 1 und 2 auf (9 und 12 Fälle) und gehen danach auf 1 (Stufe 3) und 4 (Stufe 4) zurück. Die vier Kompilierfehler in Stufe 4 gehen alle auf einen falschen relativen Importpfad der Helferdatei in UC-06 zurück.

6.3 Ergebnisse der Qualitätsbewertung (Phase 2)

6.3.1 Kontrolle der maschinellen Bewertung

6.4 Ergebnisse der Bonus-Stufe 5

6.5 GIS-spezifische Besonderheiten

7 Diskussion

7.1 Interpretation der Ergebnisse

7.2 Limitationen

7.3 Handlungsempfehlungen

8 Fazit und Ausblick

Anhang

Anhang 1: Bewertungsrubrik der Phase 2

Tabelle 3: Skalenstufen der vier Bewertungsdimensionen

Dimension	Stufe	Beschreibung
coverage	1	Testet den Use Case nicht oder etwas völlig anderes
	2	Deckt weniger als die Hälfte der Schritte ab
	3	Deckt die wesentlichen Schritte ab, einzelne Lücken
	4	Alle Schritte und erwarteten Ergebnisse abgedeckt
selector	1	Selektoren überwiegend erfunden oder falsch
	2	Mischung aus korrekten und erfundenen Selektoren
	3	Selektoren überwiegend korrekt, kleinere Abweichungen
	4	Alle Selektoren korrekt
map_interaction	n/a	Use Case erfordert keine kartenspezifische Interaktion
	1	Keine kartenspezifische Aktion oder Prüfung, obwohl der Use Case sie erfordert; ausschließlich DOM-basiert
	2	Interaktion versucht, aber fehlerhaft (erfundene Map-Model-Aufrufe, falsche Koordinaten- oder Pixellogik) oder ohne Warten auf den Kartenzustand
	3	Im Wesentlichen korrekt, kleinere Abweichungen
assertion	4	Funktional gleichwertig zum Referenztest
	1	Keine oder ausschließlich triviale Assertions
	2	Assertions vorhanden, prüfen aber nicht die Erwartungen
	3	Erwartungen im Wesentlichen geprüft, kleinere Lücken
	4	Erwartungen exakt geprüft; der Test würde bei defekter Funktion zuverlässig fehlschlagen

Anhang 2: Judge-Prompt der Phase 2

Anhang 2.1: Stufen 1–4

Listing 4: Prompt der LLM-Judge-Bewertung (Stufen 1–4)

```

1 # Phase 2: LLM-Judge-Bewertung - Stufe X
2
3 ## Rolle
4
5 Du bewertest die semantische Qualität LLM-generierter Playwright-E2E-Tests. Die
  deterministische Ausführung (Phase 1) ist bereits erfolgt. Du führst KEINE Tests
  aus und änderst KEINE Dateien außer den OUTPUT-Dateien.
```

```

6
7 ### Parameter
8
9 - STAGE_DIR: src/app/llm/tests/stage_X/
10 - PHASE1_CSV: src/app/llm/tests/stage_X/\_phase1_results.csv
11 - REFERENZTESTS: src/app/tests/manual (pro UC eine Datei)
12 - USE_CASES: src/app/llm/use_cases.md
13 - OUTPUT: src/app/llm/tests/stage_X/\_phase2_judge.csv (+ \_phase2_judge.json)
14 - MAP_UCS: uc-04, uc-06, uc-07, uc-08, uc-10 # UCs mit erforderlicher
    kartenspezifischer Interaktion
15
16 ## Vorgehen
17
18 Bewertet wird pro Use Case, nicht pro Run: Iteriere über die UCs (uc-01 bis uc-10) und
    innerhalb jedes UC über alle 50 Runs.
19
20 Pro UC:
21
22 1. Lade EINMAL die Use-Case-Beschreibung aus USE_CASES und den Referenztest aus
    REFERENZTESTS (uc-01 -> "UC-1:").
23 2. Iteriere über alle Zeilen in PHASE1_CSV mit dieser uc_id (50 Dateien) und bewerte
    jede Datei unabhängig. Lies dafür ZUERST den generierten Testcode (Datei aus
    Spalte 'file') und bilde dir ein Urteil anhand von UC und Referenztest. Ziehe
    exec_category und error_summary aus PHASE1_CSV erst DANACH als Plausibilitätscheck
    heran.
24 3. Schreibe die Ergebnisse fortlaufend nach jeder Datei, damit bei Unterbrechung
    nichts verloren geht.
25
26 Überspringe die Bewertung bei exec_category=GENERATION_ERROR: nicht bewertbar, alle
    Score-Spalten leer lassen, reasoning="GENERATION_ERROR: kein valider Testcode".
27
28 Am Ende MUSS die Zeilenzahl in \_phase2_judge.csv der in PHASE1_CSV entsprechen (abzü-
    glich Kopfzeile). Fehlt eine Datei, hole sie nach.
29
30 ## Bewertung
31
32 Bewerte vier Dimensionen in der folgenden Reihenfolge auf Skala 1-4 (
    map_interaction_score zusätzlich mit n/a). Schreibe pro Dimension IMMER ZUERST
    eine kurze Begründung (2-3 Sätze), DANN den Score.
33
34 ### coverage_score (Use-Case-Abdeckung)
35
36 1 = Testet den Use Case nicht oder etwas völlig anderes
37 2 = Deckt weniger als die Hälfte der UC-Schritte ab
38 3 = Deckt die wesentlichen Schritte ab, einzelne Lücken
39 4 = Alle Schritte und erwarteten Ergebnisse des UC abgedeckt
40
41 ### selector_score (Selektor-Korrektheit)
42
43 1 = Selektoren überwiegend erfunden/falsch (existieren nicht in der App)
44 2 = Mischung aus korrekten und erfundenen Selektoren
45 3 = Selektoren überwiegend korrekt, kleinere Abweichungen
46 4 = Alle Selektoren korrekt (stimmen mit Referenztest/App überein)
47
48 ### map_interaction_score (kartenspezifische Interaktion)
49

```

```

50 n/a = UC erfordert keine kartenspezifische Interaktion (maßgeblich ist MAP_UCS: nicht
    gelistete UCs erhalten n/a)
51 1 = Keine kartenspezifische Aktion/Prüfung, obwohl der UC sie erfordert (z. B. Canvas
    nie angeklickt, Kartenzustand nie geprüft; Aktionen und Prüfungen ausschließlich
    DOM-basiert)
52 2 = Kartenspezifische Interaktion versucht, aber fehlerhaft (erfundene Map-Model-API-
    Aufrufe, falsche Objektpfade, falsche Koordinaten-/Pixel-Logik) oder ohne Warten
    auf den Kartenzustand
53 3 = Im Wesentlichen korrekt wie im Referenztest, kleinere Abweichungen (z. B. unvollst
    ändiges Warten, schwächere Zustandsprüfung)
54 4 = Interaktion und Assertions funktional gleichwertig zum Referenztest (korrekte
    Canvas-Aktionen bzw. Map-Model-Nutzung, korrektes Warten, prüft den relevanten
    Kartenzustand)
55
56 ### assertion_score (Assertion-Angemessenheit)
57
58 1 = Keine Assertions oder ausschließlich triviale (vacuous pass möglich)
59 2 = Assertions vorhanden, prüfen aber nicht die UC-Erwartungen
60 3 = Assertions prüfen die UC-Erwartungen im Wesentlichen, kleinere Lücken
61 4 = Assertions prüfen exakt die erwarteten Ergebnisse; Test würde bei defekter
    Funktion zuverlässig fehlschlagen
62
63 ### vacuous_pass (Boolean, abgeleitet -keine eigene Ermessensentscheidung)
64
65 true genau dann, wenn exec_category=PASS UND assertion_score <= 2. Sonst false.
66
67 ## Wichtige Regeln
68
69 - Bewerte den TESTCODE, nicht das Ausführungsergebnis. Ein Test mit INFRA_FAIL oder
    COMPILE_ERROR kann trotzdem coverage_score 4 haben, wenn der Code die richtigen
    Schritte adressiert.
70 - Das Ausführungssignal (exec_category, error_summary) ist nur ein HINWEIS: "element(s)
    not found" stützt einen niedrigen selector_score, ein PASS ist Voraussetzung (
    nicht Beweis) für vacuous_pass.
71 - Der Referenztest ist die Gold-Referenz für korrekte Selektoren, kartenspezifische
    Interaktion und vollständige Abdeckung. Verlange keine identische Formulierung -
    funktional gleichwertige Lösungen sind gleichwertig zu bewerten.
72 - KONSISTENZ ist bei N=50 zentral: gleiche Fehlermuster über verschiedene Runs
    identisch bewerten. Orientiere dich beim Score strikt an den Stufendefinitionen
    oben, nicht am Vergleich mit anderen Runs.
73
74 ## Output
75
76 1. \_phase2_judge.csv -eine Zeile pro Testdatei, Spalten: stage,run,uc_id,file,
    exec_category,coverage_score,selector_score, map_interaction_score,assertion_score,
    vacuous_pass (bei GENERATION_ERROR: Score-Spalten leer; map_interaction_score kann
    "n/a" enthalten)
77 2. \_phase2_judge.json -dieselben Zeilen plus verschachteltes Feld "reasoning" mit den
    vier Begründungen (coverage, selector, map_interaction, assertion).
78
79 Keine Interpretation im Sinne der Forschungsfrage, nur Bewertung pro Datei.

```

Anhang 2.2: Stufe 5 (Self-Improvement-Loop)

Der Judge-Prompt für Stufe 5 (Self-Improvement-Loop) ist mit dem Prompt der Stufen 1 bis 4 (Anhang Anhang 2.1) identisch; die Bewertungsskalen und der Output bleiben unverändert. Abweichend sind lediglich die folgenden Punkte:

- In der Rolle wird der Ausführungskontext angepasst: Die Ausführung erfolgte während des Self-Improvement-Loops und wurde über `stage5_to_phase1.py` in die `exec_category`-Taxonomie überführt.
- In den *Wichtigen Regeln* wird ergänzt, dass auch der Loop-Verlauf nicht bewertet wird sowie eine Regel zur stufenübergreifenden Vergleichbarkeit der Maßstäbe.
- Neu hinzugefügt werden ein Abschnitt zur Besonderheit der Stufe sowie ein Hinweis zur besonderen Aufmerksamkeit beim `assertion_score` (siehe folgende Auszüge).

Listing 5: Neuer Abschnitt "Besonderheit dieser Stufe"

```
1 Die Spalte 'file' enthält den Pfad (relativ zu src/app/llm/) des final_spec --des
  Testcodes der LETZTEN Loop-Iteration. NUR diese Datei wird bewertet. Die UC-Ordner
  enthalten zusätzlich frühere Iterationen (iter-0 bis iter-n-1) und *.exec.spec.ts-
  Dateien: Das sind Harness-Artefakte --NIEMALS öffnen oder bewerten. Die Spalten '
  passed' und 'iterations_used' sind Metadaten: iterations_used ist KEIN Qualitäts-
  ts-kriterium --ein PASS nach 4 Iterationen wird identisch bewertet wie ein PASS
  nach 1.
```

Listing 6: Zusatz beim `assertion_score`

```
1 BESONDERE AUFMERKSAMKEIT in dieser Stufe: Der Loop optimiert auf PASS. Auch wenn der
  Feedback-Prompt das Abschwächen von Assertions verbietet, ist strukturell möglich,
  dass ein Test über die Iterationen hinweg weniger prüft, nur um grün zu werden.
  Prüfe daher bei exec_category=PASS besonders genau gegen die Expected Results des
  Use Case, ob die Assertions diese tatsächlich verifizieren. Die Maßstäbe der Skala
  bleiben dabei unverändert --strengere Aufmerksamkeit bedeutet nicht strengere
  Bewertung.
```

Literaturverzeichnis

- Alian, Parsa, Nashid, Noor, Shahbandeh, Mobina, Shabani, Taha, Mesbah, Ali* (2024): Feature-Driven End-To-End Test Generation, Version Number: 2, o. O., 2024, URL: <https://arxiv.org/abs/2408.01894>
- Brown, Tom B., Mann, Benjamin, Ryder, Nick, Subbiah, Melanie, Kaplan, Jared, Dhariwal, Prafulla, Neelakantan, Arvind, Shyam, Pranav, Sastry, Girish, Askell, Amanda, Agarwal, Sandhini, Herbert-Voss, Ariel, Krueger, Gretchen, Henighan, Tom, Child, Rewon, Ramesh, Aditya, Ziegler, Daniel M., Wu, Jeffrey, Winter, Clemens, Hesse, Christopher, Chen, Mark, Sigler, Eric, Litwin, Mateusz, Gray, Scott, Chess, Benjamin, Clark, Jack, Berner, Christopher, McCandlish, Sam, Radford, Alec, Sutskever, Ilya, Amodei, Dario* (2020): Language Models are Few-Shot Learners, Version Number: 4, o. O., 2020, URL: <https://arxiv.org/abs/2005.14165>
- Celik, Arda, Mahmoud, Qusay H.* (2025): A Review of Large Language Models for Automated Test Case Generation, in: Machine Learning and Knowledge Extraction, 7 (2025), Nr. 3, S. 97
- Ferreira, Margarida, Viegas, Luís, Faria, João Pascoal, Lima, Bruno* (2025): Acceptance Test Generation with Large Language Models: An Industrial Case Study, in: 2025 IEEE/ACM International Conference on Automation of Software Test (AST), 2025 IEEE/ACM International Conference on Automation of Software Test (AST), Ottawa, ON, Canada: IEEE, 2025-04-28, S. 1–11
- Hou, Xinyi, Zhao, Yanjie, Liu, Yue, Yang, Zhou, Wang, Kailong, Li, Li, Luo, Xiapu, Lo, David, Grundy, John, Wang, Haoyu* (2024): Large Language Models for Software Engineering: A Systematic Literature Review, in: ACM Trans. Softw. Eng. Methodol. 33 (2024), Nr. 8, 220:1–220:79
- Plein, Laura, Ouédraogo, Wendkûuni C., Klein, Jacques, Bissyandé, Tegawendé F.* (2023): Automatic Generation of Test Cases based on Bug Reports: a Feasibility Study with Large Language Models, Version Number: 1, o. O., 2023, URL: <https://arxiv.org/abs/2310.06320>
- Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, Polosukhin, Illia* (2017): Attention Is All You Need, Version Number: 7, o. O., 2017, URL: <https://arxiv.org/abs/1706.03762>
- Zheng, Lianmin, Chiang, Wei-Lin, Sheng, Ying, Zhuang, Siyuan, Wu, Zhanghao, Zhuang, Yonghao, Lin, Zi, Li, Zhuohan, Li, Dacheng, Xing, Eric P., Zhang, Hao, Gonzalez, Joseph E., Stoica, Ion* (2023): Judging LLM-as-a-Judge with MT-Bench and Chatbot

Arena, o. O., 2023-12-24, arXiv: 2306.05685[cs.CL], URL: <http://arxiv.org/abs/2306.05685>

Internetquellen

52°North GmbH, Arne Vogt (2025): Open Pioneer Trails: An Open-Source Framework for Map-Oriented Web Apps, Blog - 52north, <<https://blog.52north.org/2025/11/10/open-pioneer-trails/>> (2025-11-10) [Zugriff: 2026-06-15]

52°North GmbH, Matthes Rieke Managing Director (2026): Conquering new frontiers in web mapping application development, 52°North Spatial Information Research GmbH, <<https://52north.org/software/software-components/open-pioneer-trails/>> (2026-06-15) [Zugriff: 2026-06-15]

Chakra Systems (2026): Chakra Factory | Chakra UI, <<https://chakra-ui.com/docs/styling/chakra-factory>> (2026-06-18) [Zugriff: 2026-06-18]

Hugging Face (2026): Qwen/Qwen3.6-35B-A3B · Hugging Face, Qwen/Qwen3.6-35B-A3B, <<https://huggingface.co/Qwen/Qwen3.6-35B-A3B>> (2026-04-22) [Zugriff: 2026-07-27]

Microsoft (2026): Fast and reliable end-to-end testing for modern web apps | Playwright, Playwright Dokumentation, <<https://playwright.dev/>> (2026-06-03) [Zugriff: 2026-06-03]

Open Pioneer (2026): Open Pioneer GitHub, GitHub, <<https://github.com/open-pioneer>> (2026-06-15) [Zugriff: 2026-06-15]

OpenLayers (Institution) (2026): Class: CanvasImmediateRenderer (OpenLayers v10.9.0 API), <https://openlayers.org/en/latest/apidoc/module-ol_render_canvas_Immediate-CanvasImmediateRenderer.html> (2026-06-18) [Zugriff: 2026-06-18]

Poenisch, Marie (2022): Die Testpyramide, <<https://www.openknowledge.de/blog/die-testpyramide>> (2022-08-25) [Zugriff: 2026-06-03]

Qwen Team, Alibaba (2026): Qwen3.6-35B-A3B: Agentic Coding Power, Now Open to All, Qwen, Section: blog, <<https://qwenlm.github.io/blog/qwen3.6-35b-a3b/>> (2026-04-15) [Zugriff: 2026-07-27]